

ARQUITECTURA DE SOFTWARE

(TB034) Curso CALONICO - 2C 2026

Trabajo Práctico 1

Integrantes:

Lucas Nicolas Pagani	110777
Maximiliano Leonel Sorin	111688
Lucas Martín Areco Gonzalez	111203
Bustamante Rocio Camila	110573

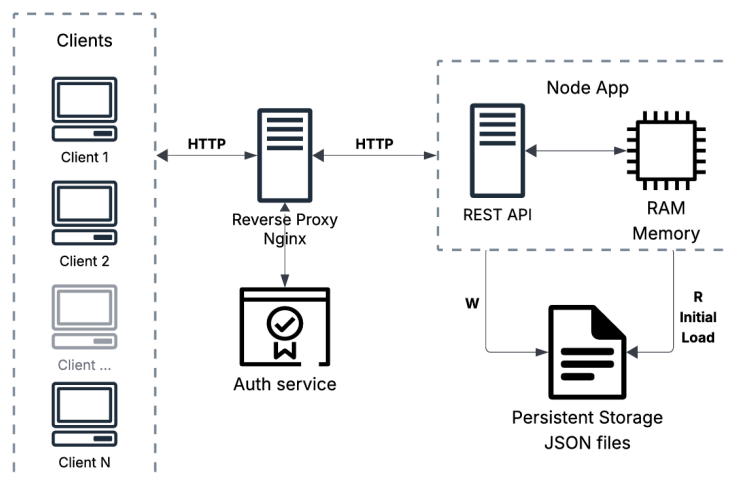
1. Análisis de la arquitectura y atributos de calidad.....	3
1.1 Contexto del negocio y arquitectura recibida.....	3
1.2 Atributos de calidad clave.....	3
1.2.1 Disponibilidad.....	3
1.2.2 Performance.....	4
1.2.3 Seguridad.....	4
1.2.4 Escalabilidad.....	5
1.3 Otros atributos de calidad.....	5
1.3.1 Usabilidad.....	5
1.3.2 Fiabilidad.....	5
1.3.3 Visibilidad.....	6
1.3.4 Manejabilidad.....	6
1.3.5 Testeabilidad.....	6
1.3.6 Portabilidad.....	6
1.3.7 Interoperabilidad.....	6
1.3.8 Simplicidad.....	7
1.3.9 Modificabilidad.....	7
1.4 Crítica General.....	7
2. Análisis del caso base.....	8
3. Tácticas aplicadas.....	16
3.1 Transacciones y Reserva anticipada.....	16
3.2 Prevención de Excepciones, Manejo de Errores y Reinicio Automático.....	24
3.3 Consistencia en las operaciones de cambio.....	27
3.4 Desacoplamiento de estado y Escalado horizontal.....	28
3.5 Operaciones atómicas para control de concurrencia.....	1
3.6 Log incremental y paginado.....	1
4. Bibliografía.....	1
4. 1 Uso de IA.....	1

1. Análisis de la arquitectura y atributos de calidad

1.1 Contexto del negocio y arquitectura recibida

arVault es una fintech que opera una billetera digital. Su servicio de cambio de monedas tiene como diferencial ofrecer la tasa más conveniente del mercado, sin gap entre compra y venta, por lo que se espera un volumen de operaciones alto y creciente. El servicio fue construido rápidamente bajo la consigna “fake it until you make it” y, luego del lanzamiento, comenzaron los reclamos de los usuarios sobre la función de cambio. Los potenciales inversores condicionan nuevos fondos a una auditoría y a una mejora del servicio. De este contexto se desprenden tres necesidades que guían la elección de los atributos de calidad: que el servicio esté disponible y haga lo que promete, que el dinero que maneja sea correcto y auditable y que soporte el crecimiento de la carga sin degradarse

Los clientes se conectan mediante requests HTTP a un reverse proxy (Nginx), que se comunica con el componente encargado de autenticar y autorizar a los clientes y, una vez autenticados, con una única instancia de la aplicación Node.js. Al iniciar, la aplicación carga en memoria el contenido de los archivos .json (cuentas, tasas y log) y, durante la ejecución, los reescribe completos de forma periódica.



1.2 Atributos de calidad clave

A partir del análisis de su arquitectura, infraestructura, código fuente, y teniendo en cuenta su contexto de rápido crecimiento esperado provocado por tener las mejores tasas de cambio, identificamos como atributos de calidad principales para este servicio la disponibilidad, la seguridad, la performance y la escalabilidad.

1.2.1 Disponibilidad

Es uno de los atributos más importantes para la reputación en el mercado financiero y el principal motivo de los reclamos, si el servicio está caído, el usuario no puede operar y seguramente busque otra alternativa. Tomamos la definición vista en clase: una falla ocurre cuando el sistema deja de brindar un servicio consistente con su especificación. Agrupamos los problemas encontrados según los tipos de falla:

- **Crash: Single Point of Failure:** Hay una sola instancia de la API y una de Nginx, sobre un único host, sin redundancia. Aunque Docker Compose permite levantar réplicas, el estado está acoplado a la instancia y el upstream de Nginx apunta a un contenedor fijo (exchange-api-1). Se analiza en la sección 3.4.
- **Crash: Errores no controlados:** Si la baseCurrency o la counterCurrency de un exchange no existen, se produce un TypeError dentro de una función async que no se captura y el proceso termina. Como ningún servicio tiene política de reinicio, la API queda caída hasta que alguien la levante a mano. Se resuelve en la sección 3.2.
- **Response Failure: race condition:** El saldo se verifica antes de las dos transferencias pero se descuenta después, así que varios exchanges simultáneos sobre la misma cuenta pueden aprobarse con el mismo saldo y dejarla en negativo. Se resuelve en la sección 3.1 y 3.5.
- **Response Failure: operaciones confirmadas y perdidas:** El estado se persiste periódicamente, así que una operación informada como exitosa puede perderse si el proceso cae antes del siguiente guardado. Se analiza en la sección 3.4.
- **Response Failure: tasas que no cumplen la promesa de negocio:** Las tasas de rates.json no son inversas entre sí, un cliente que cambia 1000 BRL a pesos y vuelve a reales termina con 1010,004 BRL, mientras que con USD pierde dinero. Se resuelve en la sección 3.3
- **Timing Failure: respuestas fuera de tiempo:** En los stress tests del caso base, al llevar la carga muy por encima de lo esperado, parte de los pedidos no se responde antes del timeout (sección 2)

1.2.2 Performance

Por el crecimiento esperado, el servicio debe procesar muchos exchanges concurrentes manteniendo tiempos de respuesta aceptables. Según las tres miradas vistas en clase:

- **Eficiencia y sensibilidad a la carga:** Los logs crecen sin límite y cada segundo se serializan enteros con JSON.stringify, que es síncrono y bloquea el event loop, a medida que acumulan operaciones, empeora el tiempo de respuesta de todos los endpoints aunque la carga sea la misma.
- **Performance percibida por el usuario (completion time):** Cada exchange hace dos transferencias en serie de 200 a 400 ms, lo que fija una base de 400 a 800 ms por operación, al que se suma el salto por Nginx.
- **Network performance (throughput y capacidad):** Todo el trabajo pasa por un único proceso, que tiene un tope que medimos en la sección 2.
- **Escritura ineficiente de archivos:** Intermitentemente, entre cada segundo y cada 5 segundos, se sobrescriben los archivos JSON para persistir los datos en la memoria, incluso si no hubo ningún cambio en los datos.

1.2.3 Seguridad

El servicio funciona para mover dinero; por ende, un saldo incorrecto es pérdida directa y lo primero que detectaría una auditoría. El enunciado aclara que la autenticación y la autorización están a cargo de un componente externo que solo cubre esos dos aspectos. Se garantiza quién puede hacer un request, pero no que su contenido sea válido. El resto queda a cargo del servicio, pero este no los protege.

- **Integridad:** El código solo verifica que los campos existan. Un exchange con monto negativo se aprueba igual, porque lo que hay que entregar también da negativo y cualquier saldo lo supera, un exchange de ARS a EUR de -2000000000 deja la cuenta ARS en -800000000. Con el monto como texto ("10"), el sistema concatena en vez de sumar y una cuenta de 60000 para a "6000010". PUT /accounts/:od/balance acepta saldos negativos y PUT /rates, tasas negativas. Además, el dinero se maneja en punto flotante.
- **No repudio:** Solo POST /exchange escribe en el log y ninguna entrada registra quién hizo la operación, así que no se puede auditar quién modificó un saldo o tasa.
- **Confidencialidad:** Ante un error no controlado, Express devuelve el stack trace completo porque la imagen no define NODE_ENV=production.
- **Disponibilidad:** Un único request con una moneda inexistente termina el proceso, y GET /log devuelve el log completo en cada llamada (sección 2)

1.2.4 Escalabilidad

Si el negocio crece como se espera, el sistema no puede incorporar recursos de manera efectiva. Verticalmente, agregar núcleos no ayuda porque Node ejecuta la lógica en un solo hilo. Horizontalmente, cada réplica tendría su propio estado y el upstream de Nginx apunta a un único contenedor

1.3 Otros atributos de calidad

Los siguientes atributos no son clave para el servicio de cambio en esta etapa, pero las decisiones del desarrollador también los afectan.

1.3.1 Usabilidad

La interfaz de usuario está tercerizada, así que evaluamos la API como interfaz para quien la consume. Algunos de los mensajes son engañosos, un exchange de ARS a ARS, o uno entre dos monedas sin tasa directa como USD a BRL responde "Not enough funds on counter currency account" cuando el problema real es que no hay tasa. Además, el servicio no valida que el cliente tenga fondos y delega ese control en la app.

1.3.2 Fiabilidad

La fiabilidad mide qué tan susceptible es el sistema a fallar por completo cuando falla una parte. En esta arquitectura casi cualquier falla parcial se convierte en una falla total.

- La API es un punto único de falla sin redundancia ni monitoreo que la reemplace o la relance. Si el proceso cae, el servicio completo deja de responder.
- El guardado reescribe cada archivo completo. Si el proceso muere durante la escritura, el .json queda truncado y al reiniciar, la carga falla; el servicio inicia igualmente, pero con las cuentas vacías y el primer exchange hace caer nuevamente el proceso.
- Si una transferencia falla después de otra exitosa, el código intenta un reintegro pero no verifica su resultado, una segunda falla dejaría el dinero del cliente sin devolver y sin registro.

1.3.3 Visibilidad

Es la capacidad de un componente de monitorear o mediar la interacción entre otros dos. Nginx ocupa esa posición entre los clientes y la API, pero solo está configurado como proxy, no filtra pedidos inválidos, no limita la tasa de pedidos y lo único que registra son sus logs de acceso por default. La API, por otro lado, no emite métricas propias ni tienen un endpoint; más allá del mensaje de inicio, sus únicos registros son `console.error` cuando falla la lectura o escritura de los archivos. Las métricas del dashboard provienen de un entorno externo, Artillery mide desde el cliente y cAdvisor los contenedores. Esto explica por qué arVault se entra de sus problemas por reclamos de los usuarios.

1.3.4 Manejabilidad

Los administradores no tienen herramientas para gestionar el sistema, sino que la administración se realiza con los mismos endpoints PUT, sin validación ni registro alguno, no hay forma de crear cuentas ni de consultar el estado del servicio y los parámetros están fijos en el código. El despliegue también es frágil, el Dockerfile copia la carpeta `state/` dentro de la imagen y el servicio `api` no monta ningún volumen, así que cada vez que se crea el contenedor el estado vuelve a los `.json` originales. Desplegar una corrección borra todas las operaciones realizadas anteriormente.

1.3.5 Testeabilidad

No hay tests automatizados y la documentación de los endpoints no especifica los payloads esperados. El servicio de transferencias está implementado dentro de `exchange.js` y siempre devuelve `true`, así que no es posible reemplazarlo por un mock, las ramas de compensación nunca se ejecutan y no es posible modificar el código. Por eso el defecto del reintegro comentado en fiabilidad queda oculto.

1.3.6 Portabilidad

El uso de contenedores suma un punto a favor ya que el sistema corre en cualquier entorno con Docker. Sin embargo, encontramos dependencias implícitas de ese entorno. El upstream de Nginx apunta al nombre fijo de `exchange-api-1`, que solo existe si el proyecto de Compose se llama `exchange`, entonces, sin el archivo que lo define, Nginx no encuentra la API y no puede iniciar. En la infraestructura de monitoreo, cAdvisor no reconoce los contenedores cuando Docker usa el almacenamiento por default en instalaciones nuevas y el panel de recursos queda vacío.

1.3.7 Interoperabilidad

La API usa HTTP y JSON, lo que facilita una integración con la app móvil tercerizada por el negocio. Sin embargo, ante un error la API responde de tres maneras diferentes, si falta un dato en el pedido, devuelve un código 400 con el mensaje "Malformed request", sin indicar cuál es el dato faltante; si la operación se rechaza por un motivo de negocio, como la falta de fondos, devuelve código 500, que corresponde con una falla del servidor; y ante un error inesperado, en lugar de JSON devuelve una página HTML con el detalle técnico del error. Como consecuencia, la app no tiene posibilidad de distinguir un rechazo por falta de fondos de una caída del servidor y no posee la capacidad de mostrar al usuario un mensaje adecuado.

1.3.8 Simplicidad

Es el atributo que el desarrollador más priorizó y es evidente: pocos componentes, sin dependencias externas de almacenamiento y un código corto y sencillo de seguir. Esa simplicidad es la que permitió llegar rápido al lanzamiento de la app y la que permitió entender el sistema en un tiempo reducido. El problema es que se logró omitiendo responsabilidades en lugar de resolverlas de forma simple.

1.3.9 Modificabilidad

La modificabilidad mide el costo de realizar cambios sobre el sistema; en este caso se encuentran un aspecto a favor y dos en contra en el desarrollo de la app.

- **Evolucionabilidad (a favor):** Todo acceso a los datos está concentrado en `state.js`, lo que facilita modificar la forma de almacenamiento (por ejemplo, reemplazar los archivos por una base de datos), la mayor parte del cambio se limita a ese archivo y el resto del código requiere ajustes menores.
- **Extensibilidad (en contra):** Agregar una moneda nueva, un cambio esperable para el tipo de servicio, no puede realizarse desde la API, hay que editar a mano `accounts.json` y `rates.json` y reconstruir la imagen. Si se intenta con `PUT/rates`, falla,; si la moneda nueva es la de destino, se guarda la tasa directa pero no la recíproca, y si es la de origen, no guarda nada. En ambos casos responde un error 500 y un detalle técnico del error. Además, el código asume que existe una única cuenta interna por moneda.
- **Configurabilidad (en contra):** Los intervalos de guardado, el puerto y el nombre del contenedor de la API están definidos directamente en los archivos, cambiarlos requiere editar el código o la configuración y volver a desplegar.

1.4 Crítica General

La decisión que más condiciona a toda la arquitectura es mantener el estado en la memoria de un único proceso y guardarlo cada cierto tiempo en archivos dentro del contenedor. Tiene sentido en el contexto en que se construyó: bajo la consigna "fake it until you make it", se priorizó la simplicidad, permitió salir rápido al mercado y ofrece lecturas muy rápidas, suficiente para mostrarle un producto a inversores. Sin embargo, de esa decisión se desprenden la mayoría de los problemas que encontramos: la pérdida de operaciones ya confirmadas, la imposibilidad de escalar horizontalmente, la race condition en el exchange y el borrado del estado en cada despliegue.

El segundo problema de fondo es que el servicio confía en todo lo que recibe. No valida tipos, signos ni monedas, acepta cualquier tasa y delega en la app el control de fondos del cliente. En un sistema que maneja dinero, esa confianza hace que cualquier error de un cliente, o cualquier abuso de alguien autorizado, termine en saldos incorrectos. El tercero es la falta de visibilidad: sin métricas propias, sin healthcheck y sin reinicio automático, el sistema no tiene forma de detectar sus propias fallas ni de recuperarse, y por eso arVault se enteró de sus problemas por reclamos de los usuarios.

Como aspecto positivo, el acceso a los datos está concentrado en `state.js`, lo que hace relativamente barato cambiar la forma de almacenamiento. En síntesis, la arquitectura recibida dio más importancia a la simplicidad y el tiempo de salida al mercado por sobre la

disponibilidad y la seguridad, que son justamente atributos claves para una aplicación de este tipo.

2. Análisis del caso base

Se realizó un relevamiento de todos los endpoints. Con el fin de evaluar los límites de performance y disponibilidad de la aplicación, se realizaron pruebas de carga sobre cada endpoint expuesto mediante Artillery. Estas incluyeron pruebas de carga (load tests) y pruebas de stress (stress tests) para analizar el comportamiento del sistema ante demanda habitual y picos altos de tráfico.

Para las pruebas de carga (load test) se utilizó una estructura de 4 fases: una fase de calentamiento breve para estabilizar el entorno antes de medir (15s, 2 req/s), una rampa de subida gradual hasta la carga objetivo (30s, de 5 a 30 req/s), una meseta sostenida representando el uso esperado en régimen normal (90s, 30 req/s), y una rampa de bajada para observar si el sistema recupera su comportamiento base tras el pico (15s, de 30 a 2 req/s)

Para las pruebas de stress se usó una estructura similar pero sin fase de bajada, llevando la rampa hasta un valor muy por encima de lo esperado para forzar la aparición de degradación o fallas. Además, se agregó una prueba de caso de uso con los endpoints GET /rates, POST /exchange y de nuevo GET /rates para revisar cómo se comporta el sistema ejecutando distintos endpoints en un mismo escenario.

GET /rates

Obtiene todos los tipos de cambio de todas las monedas en formato JSON en una sola request.

Load Test

Todas las respuestas fueron completadas exitosamente (3495 requests) a una tasa promedio de 23 req/s. El tiempo de respuesta obtuvo una mediana de 2 ms. Un 5% de las respuestas exitosas experimentaron tiempos de respuesta superiores a los 5 ms (p95) y el 1% más lento superó los 6 ms (p99), en un rango que osciló entre un mínimo de 1 ms y un máximo de 21 ms. Por otro lado, el uso de CPU en el contenedor de la API se mantuvo estable por debajo de 5% durante toda la fase sostenida, mientras que el consumo de memoria no presentó variaciones apreciables.

Stress Test (warm up: 15s, 5 req/s; ramp up: 180s, de 30 a 700 req/s; peak sustain: 30s, 700 req/s)

Se generaron un total de 86.775 solicitudes a una tasa promedio de 371 req/s, registrando una tasa de fallos de ≈10% (9385 solicitudes), correspondientes en su mayoría errores por tiempo de espera expirado.

Para las requests que lograron completarse exitosamente, el tiempo de respuesta obtuvo una mediana de 10,1 ms. La degradación de los tiempos se hace más evidente en los percentiles altos: un 5% de las respuestas exitosas experimentó demoras superiores a los 7 segundos (p95) y el 1% más lento superó los 9 segundos (p99).

El sistema responde de forma lineal en la fase inicial de ascenso. Al momento de mantener la carga, comienzan los errores y el tiempo de respuesta alcanza casi los 10 segundos. Al mismo

tiempo, tanto el contenedor de la api como de nginx sufren un aumento en el uso de CPU en los momentos de degradación, aunque no son graves.



Get /rates: load test



GET /rates: stress test

GET /accounts

Obtiene todos los saldos de cada cuenta en formato JSON en una sola request.

Load Test

Todas las respuestas fueron completadas exitosamente (3495 requests) a una tasa promedio de 24 req/s. El tiempo de respuesta obtuvo una mediana de 2 ms. Un 5% de las respuestas exitosas experimentaron tiempos de respuestas superiores a los ≈ 99 ms (p95) y el 1% más

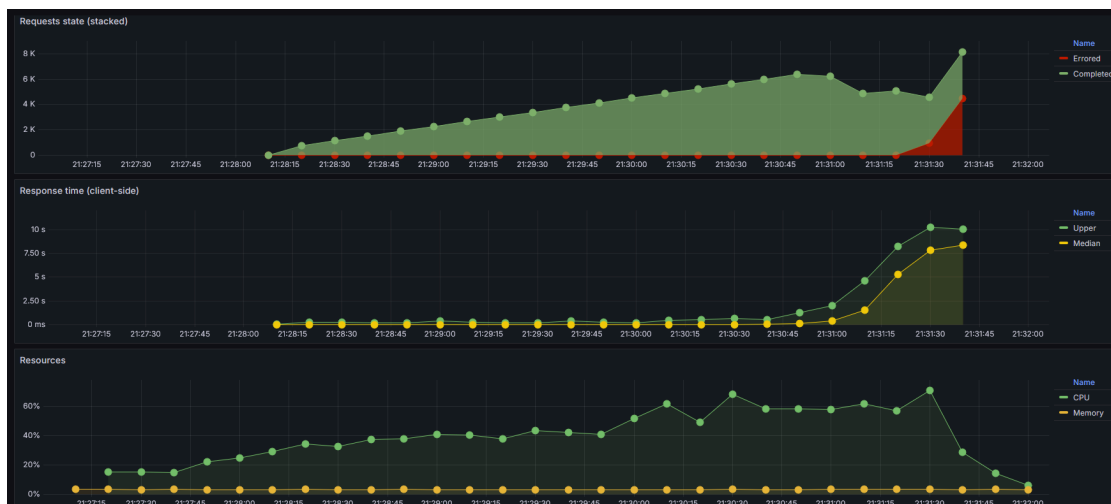
lento superó los 169 ms (p99), en un rango que osciló entre un mínimo de 1 ms y un máximo de 319 ms. Por otro lado, el uso de CPU en el contenedor de la API se mantuvo estable por debajo de 30% durante toda la fase sostenida, mientras que el consumo de memoria no presentó variaciones apreciables.

Stress test (warm up: 15s, 5 req/s, ramp up: 180s, de 30 a 700 req/s, peak sustain: 30s, 700 req/s)

Al aumentar la carga en una tasa promedio de 379 req/s se registró una tasa de fallo del $\approx 6\%$, correspondientes a errores por tiempo de espera expirado. Para las requests que lograron completarse exitosamente, el tiempo de respuesta alcanzó una mediana de 47 ms. Un 5% de las respuestas exitosas experimentó demoras superiores a los 7 segundos (p95) y el 1% más lento superó los 9,5 segundos (p99).



GET /accounts: load test



GET /accounts: stress test

POST /exchange

Permite realizar la conversión y transferencia de saldos entre cuentas de distintas divisas en base a los tipos de cambio (/rates) actuales. Para la realización de los test se configuraron las cuentas de ARS y USD con saldos elevados para que todas las requests tengan fondos suficientes.

Load Test

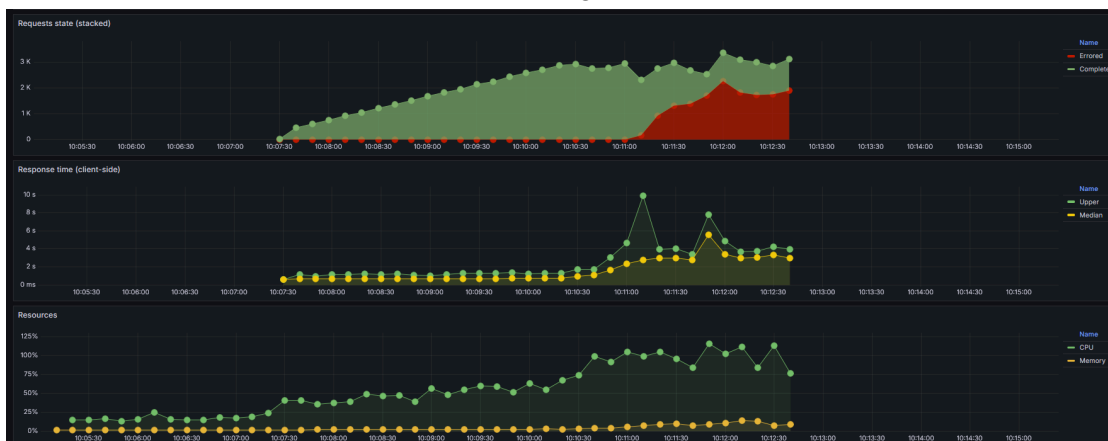
Todas las respuestas fueron completadas exitosamente a una tasa promedio de 24 req/s. Un 5% de las respuestas exitosas experimentaron tiempos de respuestas superiores a los ≈ 742 ms (p95) y el 1% más lento superó los 772 ms (p99), en un rango que osciló entre un mínimo de 405 ms y un máximo de 809 ms. Por otro lado, el uso de CPU en el contenedor de la API se mantuvo estable por debajo de 10% durante toda la fase sostenida, mientras que el consumo de memoria no presentó variaciones apreciables.

Stress test (warm up: 15s, 5 req/s, ramp up: 180s, de 30 a 300 req/s, peak sustain: 30s, 300 req/s)

Se generaron un total de 83.775 solicitudes a una tasa promedio de 224 req/s, registrando un 71,9% de respuestas exitosas y una tasa de fallos del 28,1%, correspondientes en su totalidad a errores por tiempo de espera expirado. Para el grupo de peticiones que lograron completarse exitosamente, el tiempo de respuesta obtuvo una mediana de 963,1 ms. La degradación de los tiempos se hace más evidente en los percentiles altos, un 5% de las respuestas exitosas experimentó demoras superiores a los 3,68 segundos (p95) y el 1% más lento superó los 7,11 segundos (p99), en un rango que osciló entre un mínimo de 402 ms y un máximo de 9.878 ms. El sistema responde de forma lineal en la fase inicial de ascenso. No obstante, al alcanzar el 100% de uso de CPU, los tiempos de respuesta del percentil superior registran un pico puntual de 10 segundos. A partir de ese momento, la API se estabiliza procesando un volumen constante de peticiones completadas con tiempo de respuesta en torno a los 3,5 segundos, mientras el resto no llega a procesarse a tiempo y deriva en la acumulación sostenida de solicitudes fallidas por timeout.



POST /exchange: load test



POST /exchange: stress test

GET /log (load)

Obtiene una estructura JSON con todas las transacciones realizadas por POST /exchange en una sola request. Antes de tomar mediciones sobre este endpoint, se ejecutaron varios exchanges (dos veces log population.yaml) para llenar el log.json. Por lo tanto, se considera que cada request intentó obtener aproximadamente 15 MB de datos. El test consistió en una única fase donde se ejecutaron 10 req/s durante 30 segundos.

Se generaron un total de 300 request a una tasa promedio de 10 req/s, registrando aproximadamente sólo un 17% (53 requests) de respuestas exitosas. El resto fueron correspondientes en su totalidad a errores por tiempo de espera. insuficiente. Para el grupo de requests que lograron completarse exitosamente, el tiempo de respuesta obtuvo una mediana de casi 4 segundos. El tiempo se degrada considerablemente en los percentiles altos, un 5% de las respuestas exitosas experimentó demoras superiores a los 8 segundos (p95) y el 1% más lento superó los 9 segundos (p99).

A medida que se incrementó la demanda sobre la consulta de logs, la latencia se elevó hasta generar una suba de fallos por timeout. Si bien el tiempo de respuesta registró un pico de 10 segundos y luego descendió, se debe a que la mayoría de las requests cayeron en error. Dado que la prueba se acotó a un tamaño fijo de log, el comportamiento del backend tenderá a deteriorarse sensiblemente si se incrementa el volumen de datos consultado. El sistema mantiene más del 120% de uso de CPU al momento de responder a las requests.



GET /log: load test con el log cargado

PUT /accounts/<id_account>/balance

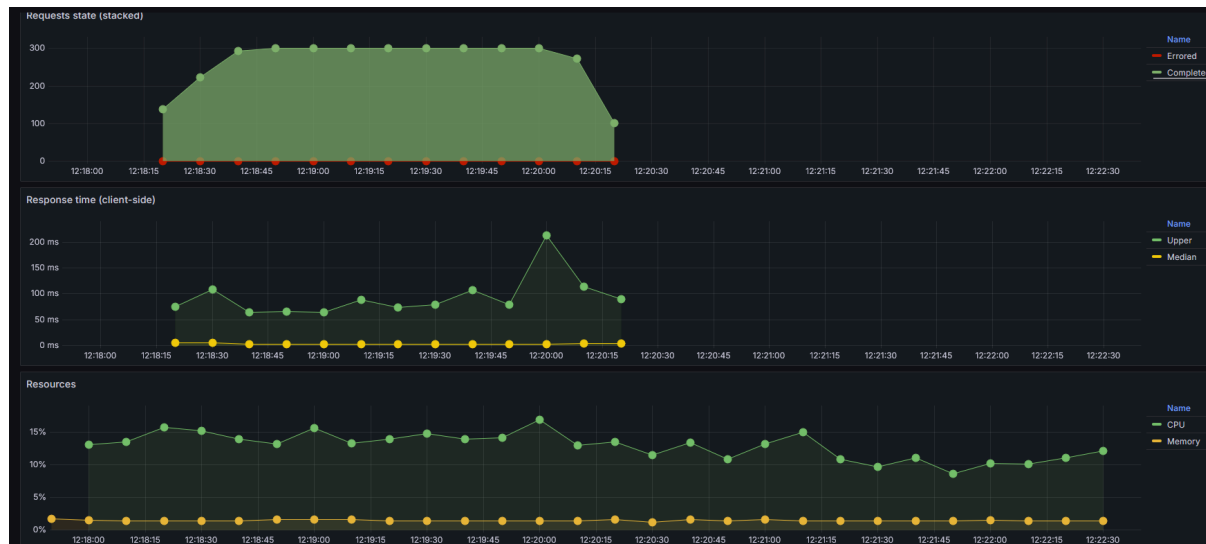
Actualiza el balance de una de una cuenta según el id.

Load Test

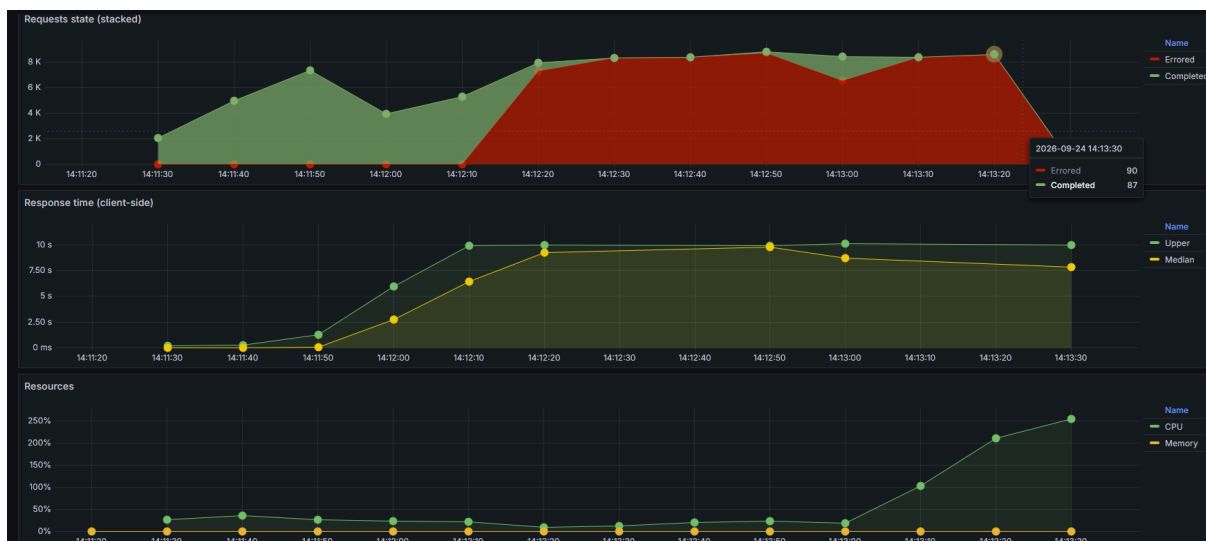
Todas las respuestas fueron completadas exitosamente a una tasa promedio de 24 req/s. Un 5% de las respuestas exitosas experimentaron tiempos de respuestas superiores a los 53 ms (p95) y el 1% más lento superó los 77,5 ms (p99). El uso de CPU en el contenedor de la API se mantuvo estable cerca del 15% durante toda la fase sostenida, mientras que el consumo de memoria no presentó variaciones apreciables.

Stress test (warm up: 15s, 5 req/s, ramp up: 180s, de 30 a 700 req/s, peak sustain: 30s, 700 req/s)

Se generaron un total de 89.595 solicitudes a una tasa promedio de 593 req/s, registrando un 70,1% de respuestas fallidas, correspondientes en su totalidad a errores por tiempo de espera expirado. Para las requests que lograron completarse exitosamente, el tiempo de respuesta obtuvo una mediana de 223 ms. La degradación de los tiempos se hace más evidente en los percentiles altos, un 5% de las respuestas exitosas experimentó demoras superiores a los 8,6 segundos (p95) y el 1% más lento superó los 9,6 segundos (p99). Finalizando la sostenida del pico, se puede notar un incremento considerable del uso de CPU en el contenedor de nginx mientras que el consumo de memoria no presentó variaciones apreciables.



PUT /balance: load test



PUT /balance: stress test

PUT /rate

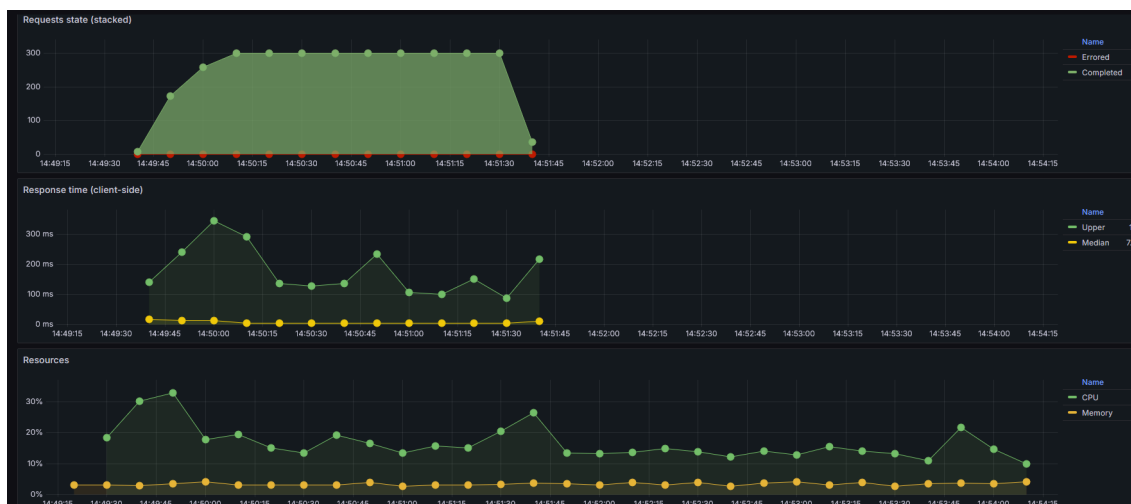
Actualiza la tasa de cambio entre dos divisas.

Load Test

Todas las respuestas fueron completadas exitosamente a una tasa promedio de 24 req/s. El tiempo de respuesta exitoso obtuvo una mediana de 2 ms. Un 5% de las respuestas exitosas experimentaron tiempos de respuestas superiores a los 71, 5 ms (p95) y el 1% más lento superó los 1625,4 ms (p99). Luego del warm up, se puede notar que se mantienen estables los niveles de recursos de API y los tiempos de respuesta.

Stress Test (warm up: 15s, 5 req/s, ramp up: 180s, de 30 a 850 req/s, peak sustain: 30s, 850 req/s)

Se generaron un total de 89.595 solicitudes a una tasa promedio de 593 req/s, registrando un 69,2% de respuestas fallidas, correspondientes en su totalidad a errores por tiempo de espera expirado. Para las requests que lograron completarse exitosamente, el tiempo de respuesta obtuvo una mediana de 340 ms. La degradación de los tiempos se hace más evidente en los percentiles altos, un 5% de las respuestas exitosas experimentó demoras superiores a los 8,8 segundos (p95) y el 1% más lento superó los 9,8 segundos (p99). Finalizando la sostenida del pico, se puede notar un incremento considerable del uso de CPU en el contenedor de nginx mientras que el consumo de memoria no presentó variaciones apreciables.



PUT /rates: load test



PUT /rates: stress test

Caso de uso (GET /rates, POST /exchange y GET /rates)

Load Test

Todas las requests se resolvieron exitosamente a una tasa promedio de 36 req/s. Desde la perspectiva del cliente, el tiempo de respuesta para completar la transacción obtuvo una

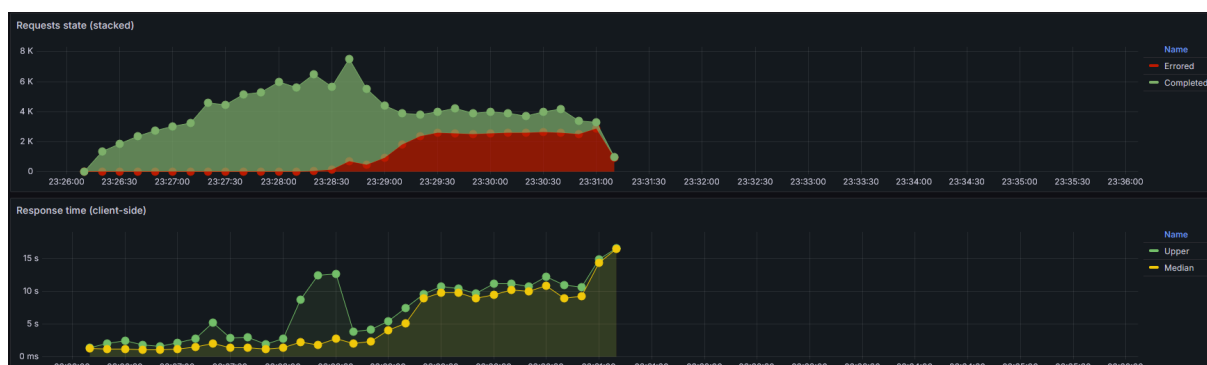
mediana de 620 ms. Un 5% experimentó más de 757 ms (p95) y el 1% más lento superó los 804 ms (p99). Estos tiempos se deben al retraso que introduce la propia transferencia de exchange, la segunda request de /rates se ejecuta recién cuando /exchange terminó su operación. El consumo de recursos permaneció estable.

Stress Test (warm up: 15s, 5 req/s, ramp up: 180s, de 30 a 300 req/s, peak sustain: 30s, 300 req/s)

A una tasa promedio de 384 req/s se evidencia que a la hora de sostener la carga comienzan las fallas, registrando una tasa de fallo del 55,2% (36.347 usuarios fallaron de 65.775 creados) debido exclusivamente a peticiones expiradas por timeout. Con respecto a la experiencia de usuario, la mediana alcanzó los 1826,2 ms mientras que el p95 rozó el límite de los 10 segundos. En los gráficos se evidencia cómo, al entrar en la meseta de máxima carga, el sistema deja de procesar tráfico de forma fluida y acumula errores por el encolamiento y bloqueo del event loop.



Caso de uso: load test



Caso de uso: stress test

3. Tácticas aplicadas

3.1 Transacciones y Reserva anticipada

Al analizar el código una de las primeras debilidades que notamos y categorizamos como grave es la posibilidad de existencia de una race condition del tipo TOC-TOU (Time Of Check - Time Of Use) al momento de ejecutar una operación de exchange:

```
//executes an exchange operation
export async function exchange(exchangeRequest) {

  //(código intermedio que no nos interesa en este momento)

  //check if we have funds on the counter currency account
  if (counterAccount.balance >= counterAmount) {
    //try to transfer from clients' base account
    if (await transfer(clientBaseAccountId, baseAccount.id, baseAmount)) {
      //try to transfer to clients' counter account
      if (
        await transfer(counterAccount.id, clientCounterAccountId,
counterAmount)
      ) {
        //all good, update balances
        baseAccount.balance += baseAmount;
        counterAccount.balance -= counterAmount;
      }
    }
  }
}
```

Como podemos ver, luego de que se chequee si la cuenta tiene los fondos suficientes, ocurren dos transferencias que dependen de un servicio el cual tiene para **cada una de ellas** una latencia de entre 200 y 400ms. Teniendo en cuenta esto, desde que se chequea si existen fondos suficientes para realizar la transferencia y efectivamente realizarlas, pueden pasar hasta 800 ms. Este tiempo, es más que suficiente para que otras transferencias simultáneas que tengan como base la misma cuenta la dejen sin fondos y, una transferencia que había pasado satisfactoriamente el filtro para saber si se tienen fondos suficientes puede dejar en negativo el balance luego de ejecutarse.

Como ya comentamos anteriormente, uno de los principales atributos de calidad que compromete esta race condition es el de disponibilidad ya que ocurren fallas del tipo Response Failure debido a que el servicio que está brindando el sistema, deja de ser consistente con su especificación (que solamente se realice la transferencia si es que se tiene los fondos suficientes en la cuenta).

Para poder evidenciarlo, vamos a someter al sistema a una prueba de stress, en la que la única operación realizada será un exchange que utilice siempre la misma cuenta de origen. Una vez realizada la carga con Artillery, consultaremos el saldo de la cuenta para verificar si quedó en un estado inválido.

Como las Race Conditions no son determinísticas, si el comportamiento no lo podemos observar en la primera ejecución, lo repetiremos varias veces.

El archivo YAML a ejecutar por artillery es el siguiente:

```
config:
  environments:
    api:
      target: 'http://localhost:5555'
      plugins:
        statsd:
          host: localhost
          port: 8125
          prefix: "artillery-api"

  pool: 50 # All HTTP requests from all virtual users will be sent over the
same connections

  phases:
    - name: 50 request en el mismo segundo
      duration: 1
      arrivalRate: 50

  scenarios:
    - name: Exchange race condition (same internal counter account)
      flow:
        - post:
            url: '/exchange'
            json:
              baseCurrency: 'ARS'
              counterCurrency: 'EUR'
              baseAccountId: 2003
              counterAccountId: 2001
              baseAmount: 10000000
```

Donde lanzaremos 50 requests de exchange en el lapso de 1 segundo con el objetivo de vaciar la cuenta de la moneda EUR (Actualmente tiene 40.000 la cuenta y el rate de ARS a EUR es de 0,00057 —> $10.000.000 * 0,00057 * 50 = 285.000$ los cuales claramente, son mayores a los 40.000 que tiene la cuenta)

Luego de la ejecución con Artillery, podemos ver que obtuvimos 32 responses codes 200 y 18 responses con error 500.

```

lucas@lucas-Z590-UD (raceConditionExchange):~/Escritorio/FIUBA/arqui/TP1-ArqSoft-404notfound/perf$ ./run-scenario.sh exchange-race api
> perf@1.0.0 artillery
> artillery run exchange-race.yaml -e api

Test run id: tmj6x_eyj7g4qar9ftxj4q4hcennntk9z4e_9r93
Phase started: 50 request en el mismo segundo (index: 0, duration: 1s) 22:18:03(-0300)

Phase completed: 50 request en el mismo segundo (index: 0, duration: 1s) 22:18:04(-0300)

-----
Metrics for period to: 22:18:10(-0300) (width: 1.516s)
-----

http.codes.200: ..... 32
http.codes.500: ..... 18

```

Este error 500 ocurre debido a que cuando se quiso hacer la operación, ya no había más fondos. El hecho de que sean 32 exchanges confirmados, ya es un mal olor debido a que si hacíamos 8 transferencias, ya se superan los 40.000 que había en la cuenta:

$$10.000.000 * 0,00057 * 8 = 45.600$$

Si hacemos una consulta al servicio para obtener las cuentas, efectivamente la cuenta quedó con fondos negativos:

```

{
  "id": 3,
  "currency": "EUR",
  "balance": -142400
}

```

Para poder resolver esto, proponemos dos soluciones diferentes:

- Implementar la táctica de transacciones haciendo que un lock serialice el flujo “chequear saldo -> dos transferencias -> debitar”.

Como bien sabemos, mejorar un QA siempre es un trade off que empeora otros. El hacer uso de esta táctica no es gratis y, si usamos esta táctica, vamos a notar un claro deterioro principalmente en dos QAs:

- Performance: Al serializar el flujo, notaremos un gran aumento en la latencia bajo concurrencia ya que los exchanges concurrentes que apunten a la misma cuenta van a dejar de correr en paralelo.

Teniendo en cuenta que en el load test del análisis del caso base, cada exchange tenía un tiempo de respuesta entre 405 ms y 809 ms, se hace difícil pensar que esta táctica aplique para el contexto de negocio que está viviendo arVault ya que en el análisis del negocio inicial, destacamos que la performance es uno de los atributos de calidad clave para el sistema. Esto es porque se espera un rápido crecimiento y una gran cantidad de operaciones debido a sus tasas de cambio competitivas, donde el servicio debe ser capaz de procesar múltiples exchanges concurrentemente manteniendo tiempos de respuesta aceptables. Es por esto, que es importante que los tiempos de respuesta no se degraden excesivamente y que el sistema pueda mantener un throughput adecuado.

- Escalabilidad: Este lock viviría en la memoria de un único proceso. Si en un futuro quisiéramos aumentar el número de instancias de las apis, cada instancia tiene su propio lock.
- Otra opción para resolver este problema es aprovechando que la API corre en node y que actualmente se tiene una única instancia de la misma, podemos implementar una idea de “reservado” que ni bien hace el chequeo, sin hacer await (y ceder el uso del CPU) ya baje el total de la cuenta. Luego, si falla el proceso de exchange en algún punto, se hace el reintegro de las operaciones que ya ocurrieron.
 - QAs que empeora:
 - Disponibilidad: El sistema podría rechazar una transferencia, lanzando un error referido a que no tiene más fondos cuando solo se trata de una reserva que no se terminará efectuando en el futuro por un error en las transferencias.
 - Ventajas:
 - No degrada la performance como el anterior enfoque ya que exchanges concurrentes que apunten a la misma cuenta pueden seguir corriendo en paralelo (El paralelo de node realmente no existe. A lo que queremos hacer referencia es que al hacer await, el proceso cede el procesador para que se ejecuten otras tareas mientras que se realizan operaciones de I/O como el método transfer en este caso que lo simula.)

No nos quedemos con el análisis verbal y puras suposiciones e implementemos estos dos cambios para ver si lo que pensamos es correcto.

Primero, vamos a chequear con la misma prueba de stress con la que mostramos la race condition, para demostrar que la misma realmente se resuelva.

De ser correcta esta suposición, aplicaremos pruebas de carga sobre la misma para ver si impacta como pensábamos en la performance cada una de estas dos tácticas (de no resolver la race condition, no tendría sentido seguir explorando estas tácticas).

Aplicando el lock y volviendo a ejecutar el mismo test de stress con artillery, podemos ver que las requests que devolvieron status code 200 son 7 y consultando el saldo, queda en un estado válido (repetimos los tests varias veces para confirmarlo por la naturaleza de las races conditions). Sin embargo, como esperábamos, subieron los response time de las response con status code 200, con una mediana de 3262.4 ms, el percentil 95 en 3678.4 ms y el 99 en 3752.7 ms.

```

http.codes.200: ..... 7
http.codes.500: ..... 43
http.downloaded_bytes: ..... 14270
http.request_rate: ..... 49/sec
http.requests: ..... 49
http.response_time:
  min: ..... 607
  max: ..... 3799
  mean: ..... 3121.9
  median: ..... 3262.4
  p95: ..... 3678.4
  p99: ..... 3752.7
http.response_time.2xx:
  min: ..... 607
  max: ..... 3799
  mean: ..... 2105.4
  median: ..... 1978.7
  p95: ..... 3197.8
  p99: ..... 3197.8
http.response_time.5xx:
  min: ..... 2663
  max: ..... 3761
  mean: ..... 3287.3
  median: ..... 3328.3
  p95: ..... 3678.4
  p99: ..... 3752.7
http.responses: ..... 50
vusers.completed: ..... 50
vusers.created: ..... 49
vusers.created_by_name.Exchange race condition (same internal counter account): 49
vusers.failed: ..... 0

```

Si ahora vemos la ejecución del mismo escenario que venimos utilizando, pero usando la segunda táctica (se descuenta al instante siguiente después de chequear), vemos que también dejó de existir la race condition, y que, como esperábamos, los tiempos de respuesta son menores:

Podemos verlo en el percentil 99 donde en el lock teníamos 3752.7ms y con la táctica de reservados, baja a 713 ms

```

http.codes.200: ..... 7
http.codes.500: ..... 43
http.downloaded_bytes: ..... 14270
http.request_rate: ..... 50/sec
http.requests: ..... 50
http.response_time:
  min: ..... 0
  max: ..... 767
  mean: ..... 93.7
  median: ..... 1
  p95: ..... 645.6
  p99: ..... 713.5
http.response_time.2xx:
  min: ..... 568
  max: ..... 767
  mean: ..... 665.6
  median: ..... 645.6
  p95: ..... 713.5
  p99: ..... 713.5
http.response_time.5xx:
  min: ..... 0
  max: ..... 1
  mean: ..... 0.7
  median: ..... 1
  p95: ..... 1
  p99: ..... 1
http.responses: ..... 50
vusers.completed: ..... 50
vusers.created: ..... 50
vusers.created_by_name.Exchange race condition (same internal counter account): 50

```

Ya vimos que el problema original de la race condition fue resuelto y nos sirvió para dar un pantallazo inicial de cómo afectaba en la performance. Pero analicemos ahora, las dos resoluciones con unas pruebas de carga como las que utilizamos a la hora de analizar el caso base. Lo dividimos en una estructura de 4 fases:

- una fase de calentamiento breve para estabilizar el entorno antes de medir
- una rampa de subida gradual hasta la carga objetivo
- una meseta sostenida representando el uso esperado en régimen normal

- una rampa de bajada para observar si el sistema recupera su comportamiento base tras el pico

Como ya la race condition la dimos por resuelta con las pruebas anteriores, vamos a usar valores bajos de las monedas para los exchanges ya que lo que nos interesa es que siempre haya dinero disponible para que no impidan los exchanges y si hay errores, que sean por la performance del programa y no por lógica del negocio.

Además, en las métricas mostraremos el volumen operado en cada moneda (compras y ventas sumadas por moneda), como así también el neto (compras suman y ventas restan) a medida que transcurre el tiempo.

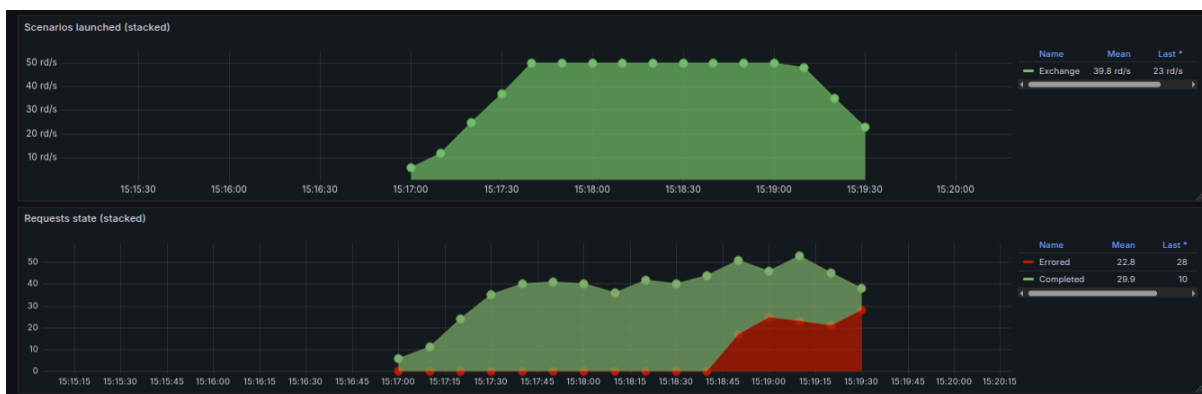
El escenario ejecutado del test de carga por lo tanto queda así:

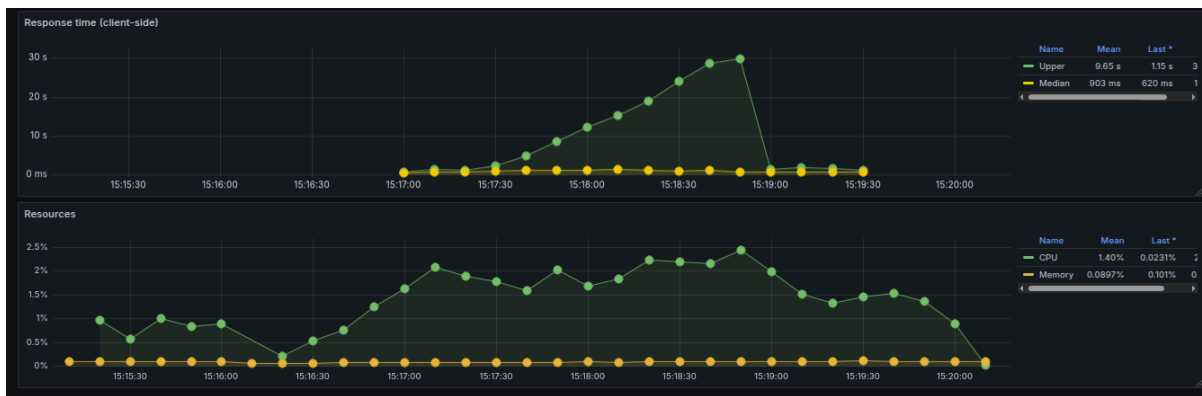
Load Test

- Warm up: 10s, 1 req/s
- Ramp up: 30s, de 1 a 5 req/s
- Sostenido: 90s, 5 req/s
- Ramp down: 30s, de 5 a 1 req/s

Los números utilizados en el test de carga, como habrán podido ver, no fueron descabellados teniendo en cuenta el análisis de negocio realizado anteriormente donde, gracias a los rates competitivos que ofrece arVault, nos imaginamos picos altos de exchanges realizados por los clientes.

En este caso, vemos como el sistema no pudo evitar que los tiempos de respuesta se degraden excesivamente ni pudo mantener un throughput adecuado durante 90 segundos a 5 req por segundo. Esto se debe, como comentamos anteriormente, a la serialización de acceso a los datos que se encuentran en memoria. Como cada transferencia puede llegar a durar 800 ms, en la meseta se iban encolando los diferentes exchanges solicitando accesos a los recursos compartidos llegando a un máximo de response time de 30 segundos antes de que se empiecen a lanzar TIMEOUTS.





Podemos ver como los recursos (CPU y memoria) no presentaron grandes dificultades en las pruebas de carga siendo desperdiciados mientras se lanzaban timeouts.

Si bien es verdad que esta táctica resolvió el problema de la race condition, la degradación de la performance fue muy grande y no acompaña las reglas de negocio definidas anteriormente. Por último, si revisamos el neto y los volúmenes de las monedas, podemos ver tres cosas interesantes en el escenario que acabamos de ejecutar.

Lo primero, es ver como la moneda ARS es dominante en ambos gráficos dejando despreciables los valores de las otras monedas. Esto ocurrió principalmente por dos razones:

- En todos los exchanges de cualquier moneda se encuentra presente ARS
- Pusimos en el escenario que todos los exchanges sean de valor 10. El problema es que 10usd corresponden a 15.130ARS mientras que 10ARS corresponden a 0,0066 usd. Si tenemos en cuenta que los valores van a ser similares para las demás monedas, es entendible que los movimientos de ARS sean mayores

Otra cosa que podemos ver, es cómo se parece mucho el dashboard del neto de ARS con el de movimientos. Esto está relacionado con el punto anterior y es que, como siempre se usaron exchanges de 10, una salida de 10 ARS es mucho menor a que una entrada de cualquier moneda de valor 10 convertida a ARS.

El último y quizá más interesante hallazgo, es como los movimientos son lineales y se empiezan a “planchar” cuando el programa empieza a tirar TIMEOUTS. La condición de ser lineales, es debido a la serialización de los exchanges y el planchado se debe a que solo se guarda la métrica si las transferencias fueron exitosas.



Ejecutemos ahora, el mismo test de carga para la implementación del reservado. Recordemos como era el test:

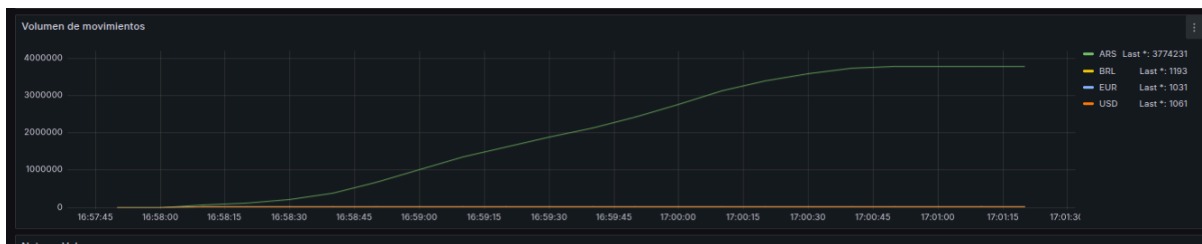
- Warm up: 10s, 1 req/s
- Ramp up: 30s, de 1 a 5 req/s
- Sostenido: 90s, 5 req/s
- Ramp down: 30s, de 5 a 1 req/s

Lo primero que podemos ver en comparación con la anterior táctica, es que no hubo ninguna request que haya tenido errores. Además, los tiempos de respuesta bajaron significativamente teniendo un máximo de respuesta de 793 ms.

Esto coincide con lo esperado cuando analizamos las opciones, debido a que a diferencia de usar el lock, esta alternativa no degrada la performance como el anterior enfoque ya que exchanges concurrentes que apunten a la misma cuenta pueden seguir corriendo en paralelo.



Por último, si revisamos el volumen de movimientos, podemos ver como en los ARS, siguió creciendo linealmente hasta la fase "Ramp Down" donde empezó a decrecer hasta el final de la ejecución que quedó en una meseta



3.2 Prevención de Excepciones, Manejo de Errores y Reinicio Automático

Como describimos en 1.3, un exchange con una moneda inexistente termina el proceso de la API y, sin política de reinicio, el servicio permanece caído. Como objetivo, nos propusimos que ante una caída el servicio vuelva sin intervención humana en menos de 30 segundos.

Antes del pedido, GET /rates responde 200, después, la API pasa a Exited y todos los endpoints responden 502. Para medirlo, armamos una prueba (prueba-disponibilidad.sh) que envía 10 GET /rates por segundo durante 90 segundos y, a los 30 segundos, el pedido inválido, midiendo cuánto tarda el servicio en volver a responder 200.

```
===== 1. ANTES: GET /rates =====
HTTP 200
===== 2. POST /exchange con moneda inexistente (XXX) =====
HTTP 502
===== 3. Estado del contenedor de la API =====
exchange-api-1 Exited (1) 3 seconds ago
===== 4. DESPUES: GET /rates =====
HTTP 502
===== 5. Error en el log de la API =====
api-1 | at Route.dispatch (/opt/app/node_modules/express/lib/router/route.js:119:3)
api-1 | at Layer.handle [as handle_request] (/opt/app/node_modules/express/lib/router/layer.js:95:5)
api-1 | at /opt/app/node_modules/express/lib/router/index.js:284:15
api-1 | at router.process_params (/opt/app/node_modules/express/lib/router/index.js:346:12)
api-1 | at next (/opt/app/node_modules/express/lib/router/index.js:280:10)
api-1 | at /opt/app/node_modules/body-parser/lib/read.js:137:5
api-1 | Node.js v24.21.0
```

Para medir el impacto armamos una prueba de disponibilidad (disponibilidad.yaml y prueba-disponibilidad.sh). Durante 90 segundos se envían 10 pedidos por segundo a GET /rates, un endpoint liviano que no depende del servicio de transferencias, así que cualquier error que aparezca se debe a la disponibilidad y no a la performance. A los 30 segundos de iniciada la carga se envía el pedido con la moneda inexistente, y el script mide cuánto tiempo el servicio deja de responder y si se recupera solo.

En el caso base no se recuperó en ningún momento, de los 900 pedidos de la carga, los 297 enviados antes del pedido inválido respondieron bien y los 603 siguientes fallaron con 502. En el dashboard se ve que las requests completadas caen a cero a los 30 segundos y no vuelven, y que el panel de recursos deja de mostrar datos porque el contenedor de la API se detuvo. Los pedidos fallidos responden 502, que el panel Requests state no marca como errores, por eso la falla se ve en la caída de las requests completadas.

Aplicamos tres tácticas de disponibilidad, cada una actúa en un momento distinto de la falla:

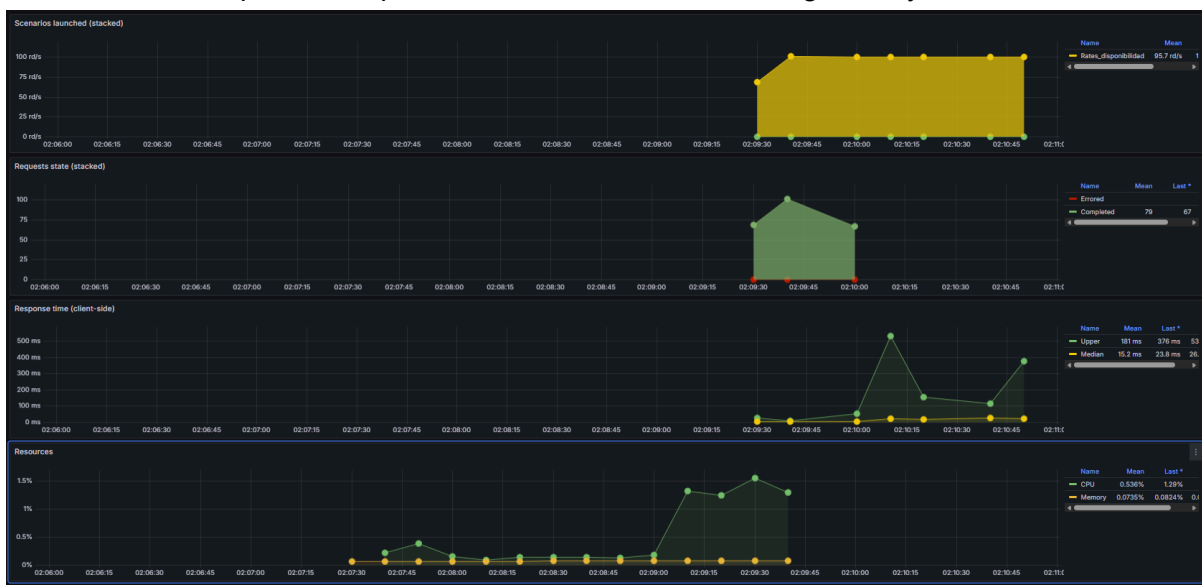
- Exception Prevention: Validando cada pedido antes de que llegue a la lógica del negocio y respondiendo 400
- Exception Handling: Capturamos cualquier error inesperado para que falle ese pedido enviando un error 500 y no todo el proceso
- Monitor con reinicio automático: Con la política de restart: unless-stopped de Docker. Si el proceso de la API termina por cualquier motivo, Docker lo detecta y vuelve a levantar el contenedor. No evita la caída, pero reduce el tiempo fuera de servicio.

Repetimos la prueba de disponibilidad en las 3 configuraciones de la app

Configuración	Pedidos Fallidos	Tiempo fuera de servicio	¿Se recupera solo?
Caso Base	603 de 900	Más de 60 segundos	No, requiere intervención manual
Sólo reinicio automático	2 de 900	1 segundo	Si
Validación + reinicio automático	0 de 900	0	No llega a caerse

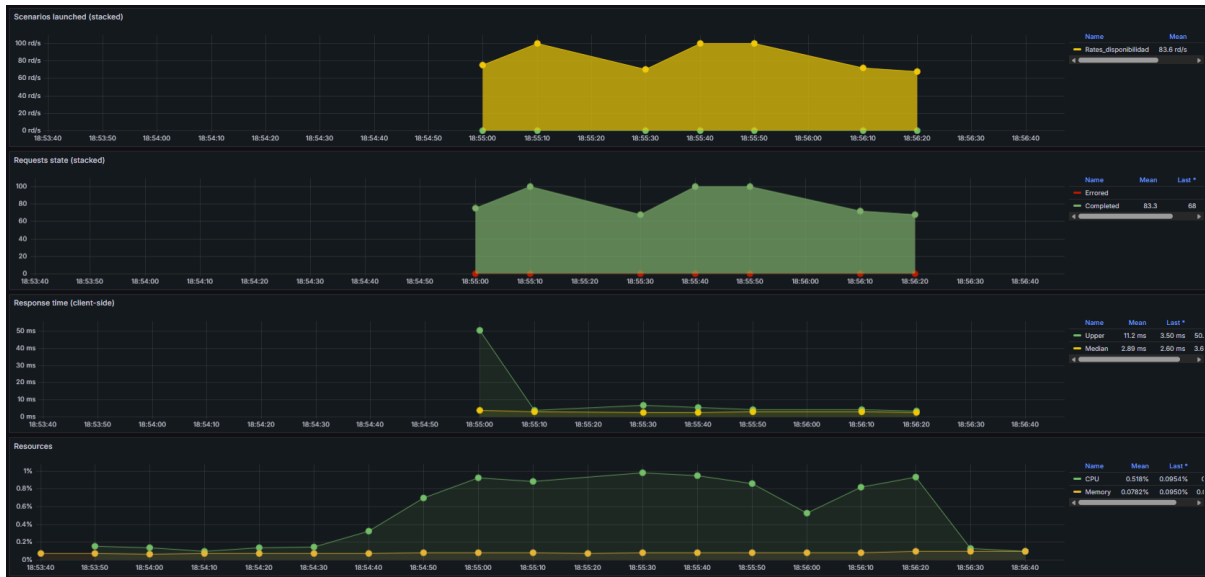
Sólo reinicio automático: el pedido inválido tira abajo la API (responde 502 y el contenedor registra un reinicio), pero Docker la vuelve a levantar y el servicio responde otra vez en 1 segundo. Solo 2 de los 900 pedidos fallaron, y en el dashboard el corte casi no se nota.

Caso base: las requests completadas caen a cero a los 30 segundos y no vuelven



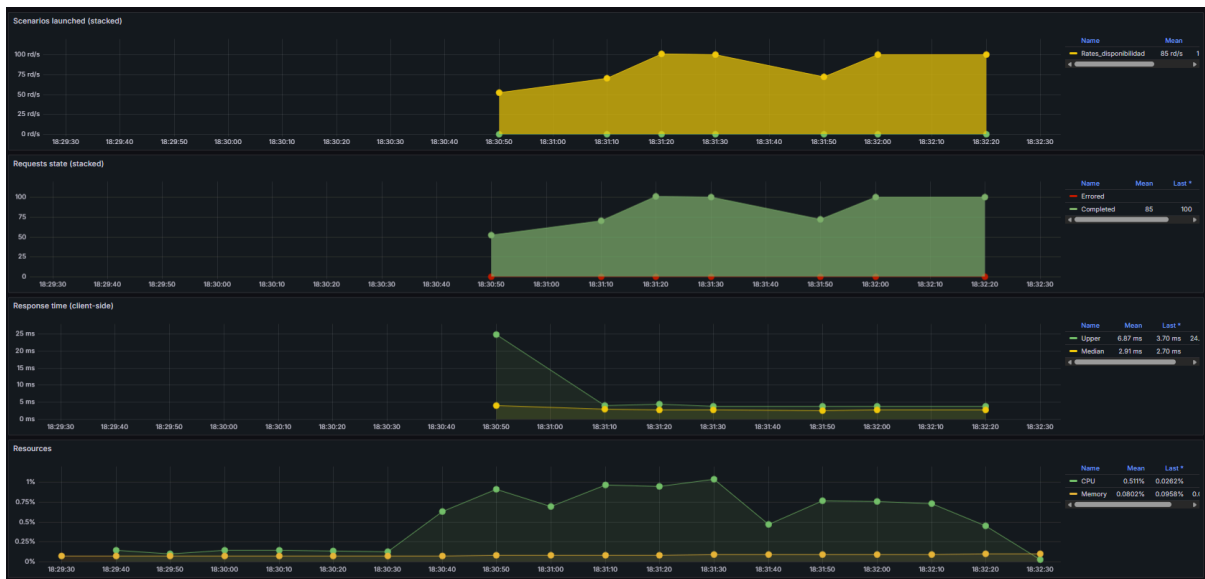
```
===== RESUMEN =====
GET /rates antes del pedido inválido: 200
Respuesta al pedido inválido: 502 (000 = sin respuesta)
RESULTADO: el servicio NO volvió en 60 s (requiere intervención manual)
Reinicios del contenedor de la API: 0
Pedidos de la carga (Artillery): 900 enviados, 297 OK, 603 fallidos
=====
```

Solo reinicio, el corte dura un segundo y casi no se evidencia



```
===== RESUMEN =====
GET /rates antes del pedido inválido: 200
Respuesta al pedido inválido: 502 (000 = sin respuesta)
RESULTADO: el servicio volvió a responder a los 1.0 s (tiempo fuera de servicio)
Reinicios del contenedor de la API: 1
Pedidos de la carga (Artillery): 900 enviados, 898 OK, 2 fallidos
=====
```

Validación + reinicio automático: el pedido invalido recibe un 400, el proceso no se cae



```
===== RESUMEN =====
GET /rates antes del pedido inválido: 200
Respuesta al pedido inválido: 400 (000 = sin respuesta)
RESULTADO: el servicio nunca dejó de responder (tiempo fuera de servicio: 0 s)
Reinicios del contenedor de la API: 0
Pedidos de la carga (Artillery): 900 enviados, 900 OK, 0 fallidos
=====
```

En relación al impacto en otros atributos, mejoran la seguridad (se bloquean montos negativos y montos como texto, y no se expone el stack trace) y la interoperabilidad (los errores responden en JSON con un mensaje claro). Como tradeoff, el reinicio recupera el servicio pero no el estado, así que se pierden las operaciones de la ventana de persistencia, y puede ocultar que hubo una falla si no se revisa la cantidad de reinicios del contenedor. La modificabilidad tiene un costo bajo: hay más código en app.js, pero las reglas se basan en el estado del sistema.

3.3 Consistencia en las operaciones de cambio

Como describimos en 1.2.1, las tasas de rates.json no son inversas exactas entre sí, un cliente que cambia 1000 BRL a pesos y los vuelve a cambiar a reales termina con 1010,004 BRL, y con USD el cliente pierde dinero en el camino. El servicio promete al cliente una conversión reversible al tipo de cambio publicado, y no la estaba cumpliendo.

Para medirlo armamos prueba-tasas.sh que consulta GET /rates, convierte 1000 unidades de BRL, EUR y USD a ARS y de vuelta a la moneda original comparando el resultado contra las 1000 unidades de inicio, y hace un PUT /rates fijando USD a ARS a mano para revisar qué tasa inversa queda guardada.

```
=====
1. Tasas cargadas
=====
{"ARS":{"BRL":0.0034,"EUR":0.00057,"USD":0.00066},"BRL":{"ARS":297.06},"EUR":{"ARS":1761},"USD":{"ARS":1513}}
=====
2. Ida y vuelta con 1000 unidades de cada moneda
=====
Moneda Recibe en ARS      Vuelve a recibir  Diferencia
BRL     297060           1010.003999999999 10.0040
EUR     1761000          1003.77              3.7700
USD     1513000          998.58              -1.4200
=====
3. PUT /rates USD->ARS = 1513: ¿qué inversa guarda?
=====
Inversa guardada ARS->USD: 0.00066 (valor exacto: 0.0006609385327164573)
Ida y vuelta con esa tasa: 1513 x 0.00066 = 0.99858
```

En el caso base, la ida y vuelta posee una diferencia de 10.0040 BRL, 3,7700 EUR y -1.4200 USD cada 1000 unidades cambiadas, con BRL y EUR el cliente termina con más de lo que puso, con USD termina con menos. En ambos casos el sistema no cumple lo que promete al cliente.

Aplicamos dos correcciones, cada una en un momento diferente. Al iniciar el servicio, se recalculan todas las tasas inversas a partir de las tasas directas cargadas (normalizeRates), en vez de mantener ambos lados escritos a mano en rates.json, que es donde se habían desincronizado.

En cada PUT /rates se dejó de redondear la tasa inversa a 5 decimales antes de guardarla. Ese redondeo intermedio perdía precisión en cada actualización manual y terminaba reproduciendo el mismo problema con el tiempo.

```

=====
1. Tasas cargadas
=====
{"ARS":{"BRL":0.0033663233016898945,"EUR":0.0005678591709256105,"USD":0.0006609385327164573},"BRL":{"ARS":297.06},"EUR":{"ARS":1761},"USD":{"ARS":1513}}
=====

2. Ida y vuelta con 1000 unidades de cada moneda
=====
Moneda Recibe en ARS      Vuelve a recibir  Diferencia
BRL    297060             1000                0.0000
EUR    1761000            1000                0.0000
USD    1513000            1000                0.0000
=====

3. PUT /rates USD->ARS = 1513: ¿qué inversa guarda?
=====
Inversa guardada ARS->USD: 0.0006609385327164573 (valor exacto: 0.0006609385327164573)
Ida y vuelta con esa tasa: 1513 x 0.0006609385327164573 = 1
=====

```

En relación al impacto en otros atributos, la corrección no tiene costo de performance relevante, `normalizeRates` recorre una sola vez al iniciar una lista corta. Mejora la fiabilidad y la testeabilidad, porque `prueba-tasas.sh` siempre da el mismo resultado sin importar cuantas veces corra el programa, no hay necesidad de reiniciar el contenedor entre ejecución y ejecución.

3.4 Desacoplamiento de estado y Escalado horizontal

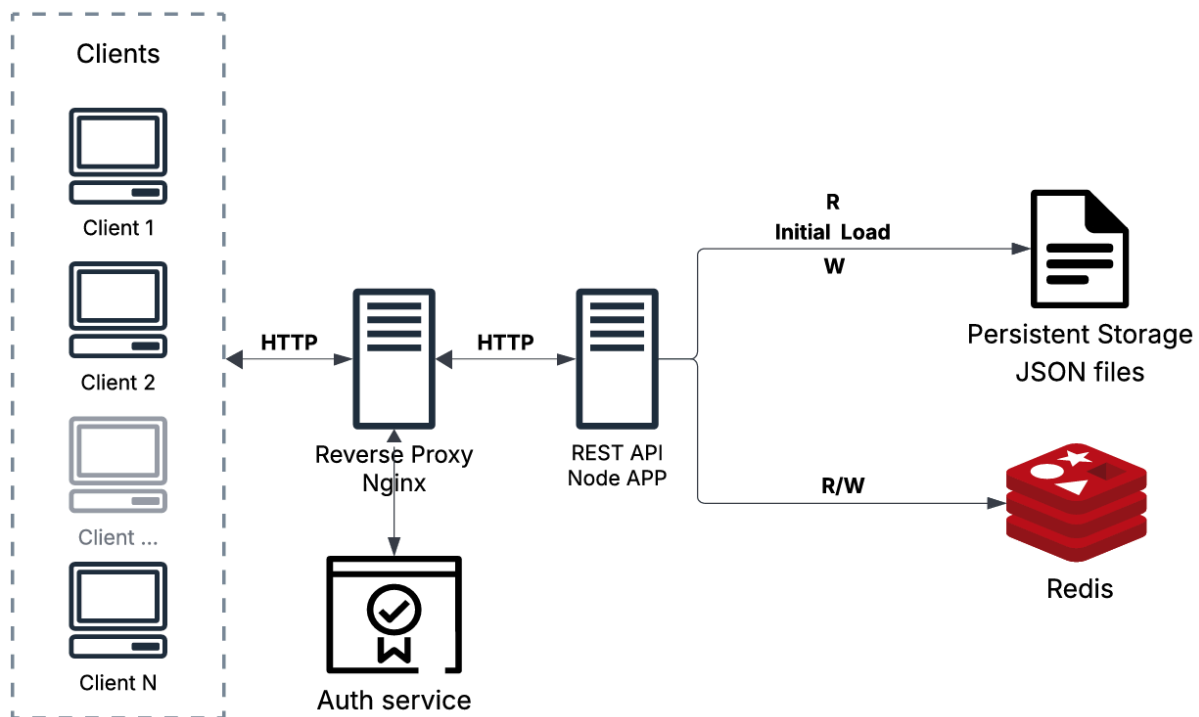
Una limitación que notamos a lo largo del análisis fue la imposibilidad de escalar horizontalmente las instancias de la API debido a que el estado de las cuentas está completamente acoplado a la única instancia. Es por esto que, si levantamos réplicas en el Docker Compose, cada una va a manejar información diferente y no va a ser determinístico el funcionamiento del sistema.

Una alternativa que vamos a analizar es agregar una base de datos (vamos a analizar la implementación del motor Redis ya que fue explícitamente propuesto por el fundador de arVault) que se encargue de manejar el estado haciendo que el guardado de datos no dependa de cada instancia de la API.

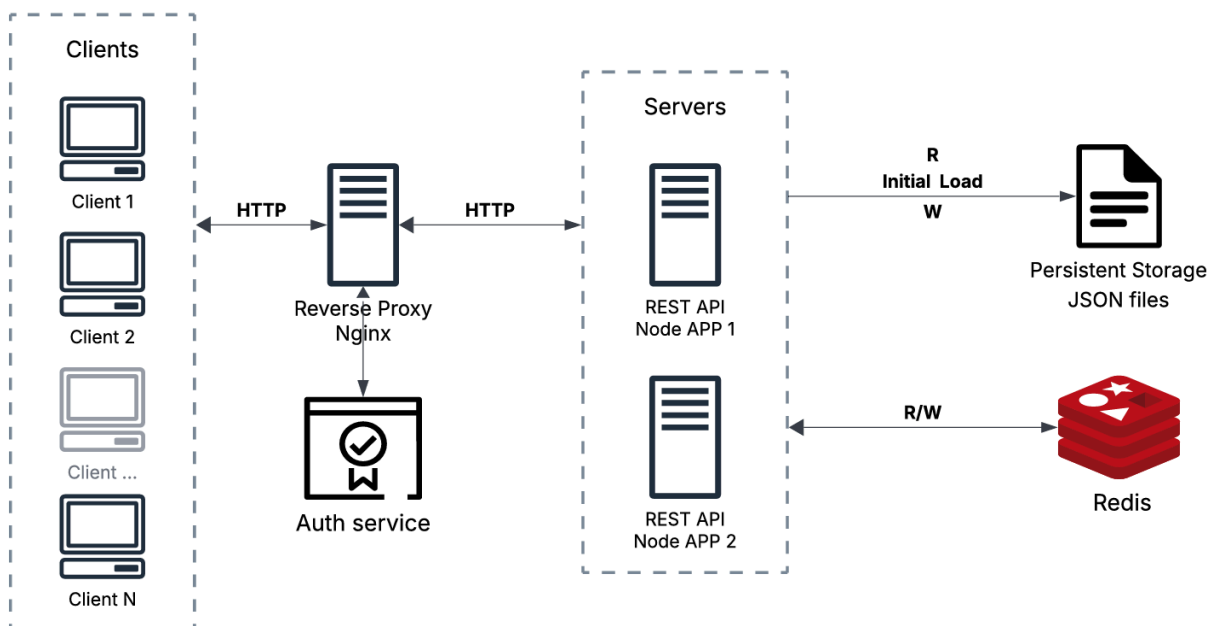
Esperamos que esta modificación mejore atributos de calidad como la escalabilidad al permitir agregar instancias de la API que compartan una misma fuente de verdad, y la disponibilidad, al desacoplar el estado del ciclo de vida de una instancia particular.

A su vez, creemos que la incorporación de Redis puede introducir un costo adicional de acceso al estado y afectar la performance, además de incorporar un nuevo punto único de falla.

Aca vemos un diagrama de una única instancia de API con Redis:



Más de una instancia de API guardando el estado en una BDD Redis como única fuente de verdad:



Otra de las razones principales por la que se ve necesario introducir una base de datos es que los logs crecen sin límite y cada 1 segundo se serializan enteros con `JSON.stringify`, que al ser síncrono, bloquea el event loop mientras está funcionando.

Además, a medida que pasa el tiempo, se acumulan cada vez más operaciones y el event loop se bloquea cada vez más empeorando el tiempo de respuesta de todos los endpoints, aunque la carga sea la misma.

Como desventaja, hemos movido el lugar donde se encuentra el punto único de falla, ya que ahora lo tenemos en la instancia de Redis.

Antes de arrancar con la implementación de redis, destacar que para no acoplar tácticas, las estamos probando por separado. Por lo que ahora los tests que estamos por realizar, NO tienen implementado las tácticas realizadas anteriormente (como por ejemplo ninguna de las race conditions fue resuelta en los próximos escenarios. Más adelante veremos cómo aplica ese problema para nuestra arquitectura migrada a una BDD).

El primer ejemplo que vamos a realizar, será para confirmar la hipótesis de que actualmente, levantando 3 instancias de la API, no van a ser determinísticas porque cada una va a tener su propio estado. Para esto, vamos a ejecutar el mismo test de carga que usamos para analizar el caso base:

Load Test

- Warm up: 15s, 2 req/s
- Ramp up: 30s, de 5 a 30 req/s
- Sostenido: 90s, 30 req/s
- Ramp down: 15s, de 30 a 2 req/s

Luego de ejecutarlo y cuando ya no haya operaciones de exchange activas, vamos a hacer algunas consultas del estado de las cuentas a la API. Si nuestra hipótesis es correcta, las mismas nos traerán diferentes resultados dependiendo de la instancia del servidor que escoja nginx para ejecutar la consulta.

Por último, agregamos dos escenarios de exchange a ejecutarse con el mismo peso:

- 10 USD a ARS
- 15151.51 ARS a USD

El motivo principal de esto, es que sean exchanges “contrarios” para que ninguna de las dos cuentas se quede sin fondos. El segundo motivo es poder ver cómo impacta esto en los dashboards de las monedas.

Luego de ejecutar el escenario detallado anteriormente, podemos confirmar la hipótesis planteada viendo que al ejecutar varias veces el get a las cuentas, podemos obtener alguno de estos 3 valores distintos:

The image displays three side-by-side screenshots of JSON API responses, each showing the state of four accounts (id 1 to 4) for different currencies. The accounts are: id 1 (ARS), id 2 (USD), id 3 (EUR), and id 4 (BRL). The balances shown in the three screenshots are different, illustrating the non-deterministic nature of the system when multiple instances are accessed simultaneously.

id	currency	balance
1	ARS	120148817.88000315
2	USD	59910.00199919811
3	EUR	40000
4	BRL	60000

id	currency	balance
1	ARS	120027691.84000313
2	USD	59990.00198559812
3	EUR	40000
4	BRL	60000

id	currency	balance
1	ARS	119906501.2700031
2	USD	60070.001961798145
3	EUR	40000
4	BRL	60000

En todas coinciden los valores de EUR y BRL ya que en ningún escenario fue modificado ninguna de estas dos monedas.

Veamos ahora, cómo impactan estos escenarios en los gráficos de las monedas. Lo primero que vemos es al igual que en los gráficos anteriores, como aunque hayamos balanceado los valores, el conteo de las unidades sigue siendo muchísimo más grande para ARS que para USD. Esto es debido a que el ARS tiene una denominación mucho más chica que el USD:

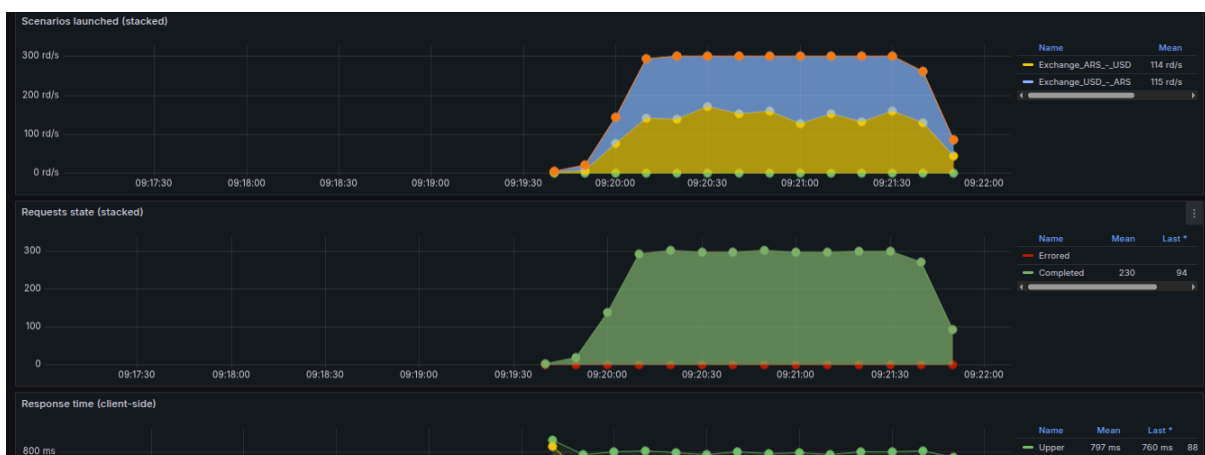


La línea naranja es la que representa al USD mientras que la verde representa el ARS. Para que sea más significativo el dashboard de los movimientos netos y los volúmenes, podríamos normalizar cada movimiento a una moneda común o, más interesante, veamos que pasa si solo mostramos el mismo gráfico de la misma corrida del mismo escenario pero solo mostrando los USD (línea naranja) y luego solo mostrando los ARS (línea verde):



Podemos ver que el hecho de que los valores intercambiados entre monedas del exchange estén balanceados, hace que la curva del gráfico (aunque a diferentes escalas), sean muy parecidas. Y esto era el comportamiento esperado para que los valores netos de las cuentas están oscilando cerca del cero así no nos quedamos sin dinero y empezábamos a recibir errores por lógica de negocio que no nos interesa para este análisis específico.

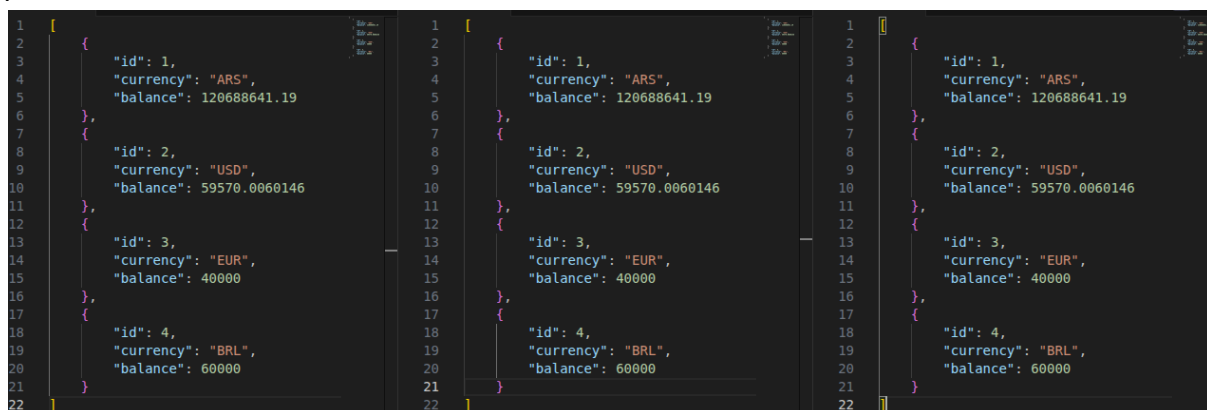
Por último, mostrar que todas las requests fueron completadas con tiempos de respuesta similares a los que venimos analizando en estos tests de carga y que como esperábamos, los escenarios lanzados fueron casi los mismos:



Si ahora realizamos esta misma ejecución del mismo escenario, el mismo Artillery YAML, misma duración, mismas tasas, mismos exchanges y mismas tres APIs pero implementando el guardado del estado en la base de datos redis, podremos ver como las distintas instancias dejan de manejar copias independientes del estado y pasan a utilizar una única fuente de información compartida.

De esta forma, lo que esperamos es que luego de finalizar la prueba de carga, las sucesivas consultas de balance devuelvan siempre el mismo resultado independientemente de cuál de las tres instancias sea seleccionada por Nginx para atender la solicitud. Si esto es así, pudimos comprobar que el hecho de hacer externo a las instancias de la API el estado, nos permitió escalar horizontalmente los nodos mejorando QAs como la escalabilidad y la disponibilidad. Sin embargo (vamos a detallarlo en otro análisis), creemos que la necesidad de utilizar una base de datos, puede haber empeorado la performance ya que los datos dejan de vivir en la memoria del programa.

Luego de hacer los cambios en el código para hacer uso de redis y ejecutar el escenario de artillery, podemos ver como unificamos el estado del programa donde aunque realicemos varias consultas del balance, siempre recibimos el mismo resultado ya que tenemos una sola fuente de verdad. Sin embargo, como comentamos anteriormente, hemos creado un nuevo punto único de falla.



```
1 [{
2   {
3     "id": 1,
4     "currency": "ARS",
5     "balance": 120688641.19
6   },
7   {
8     "id": 2,
9     "currency": "USD",
10    "balance": 59570.0060146
11  },
12  {
13    "id": 3,
14    "currency": "EUR",
15    "balance": 40000
16  },
17  {
18    "id": 4,
19    "currency": "BRL",
20    "balance": 60000
21  }
22 }]
```

Ya vimos que la implementación de Redis nos permitió escalar horizontalmente los nodos del servidor. ¿Pero cómo impacta esto en la performance del sistema?

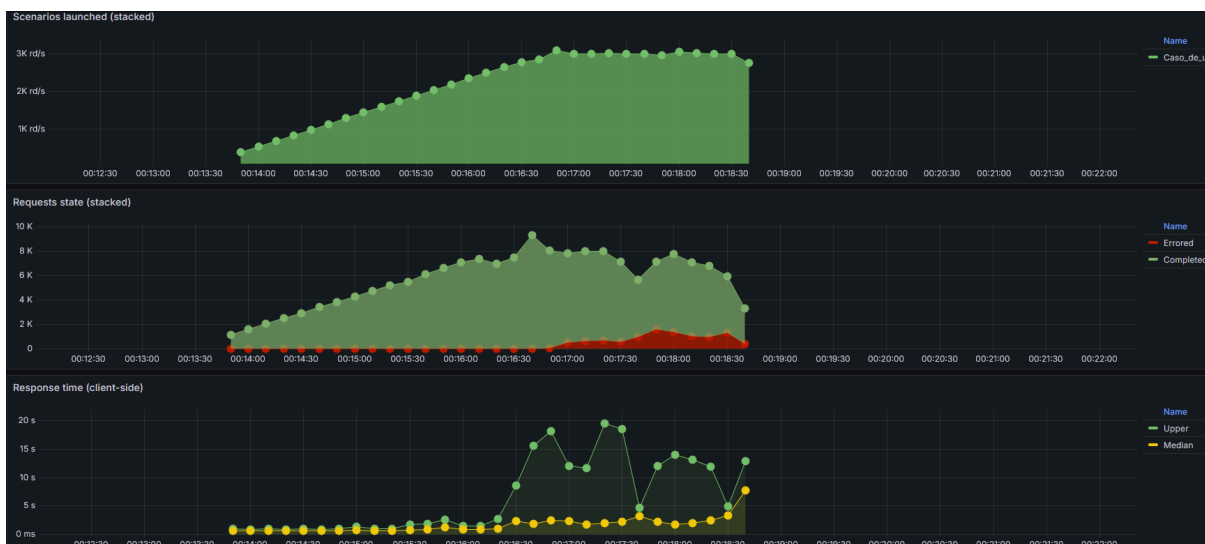
Se realizó un load test y un stress test sobre el mismo caso de uso que se aplicó al caso base.

Load Test



Al igual que en el caso base, todas las peticiones se resolvieron exitosamente. Sin embargo, el throughput promedio se duplicó, pasando de 36 req/s a 72 req/s, lo que confirma que delegar la persistencia a Redis libera a Node.js del costo de I/O sobre archivos JSON. El tiempo de procesamiento por request registró una mediana de 7 ms. Desde la perspectiva del cliente, la mediana del flujo completo se ubicó en 632,8 ms ligeramente superior al caso base debido al salto de red adicional hacia Redis, manteniendo un p95 de 772,9 ms y un p99 de 820,7 ms. El consumo de CPU y memoria fue estable.

Stress Test



A una tasa promedio de 551 req/s, la incorporación de Redis logró reducir la tasa de fallo al 17,8% (11.749 usuarios fallaron de 65.775 creados), mejorando notablemente frente al 55,2% del caso base y elevando las requests exitosas a más de 166.000. Con respecto a la experiencia de usuario, la mediana disminuyó a 1.587,9 ms, mientras que el p95 experimentó una mejora más drástica, cayendo de los 10 segundos del caso base a 1.826,6 ms. Si bien delegar la persistencia a memoria liberó la API del cuello de botella de I/O de disco, la presencia de errores por timeout demuestra que la capacidad de procesamiento de una única

instancia sigue siendo insuficiente ante un pico sostenido de usuarios, evidenciando la necesidad del escalado horizontal.

Como consecuencia, la arquitectura evolucionó hacia un escalado horizontal. Se ejecutaron de nuevo las pruebas de carga sobre un escenario conformado por tres instancias de la API, utilizando Nginx como Load Balancer bajo el mecanismo Round Robin.

Load Test



Todas las peticiones se resolvieron exitosamente a la misma tasa promedio de 72 req/s. Al repartir el tráfico entre tres instancias independientes, la carga individual disminuyó y la mediana de procesamiento interno en el servidor bajó de 7 ms a 3 ms. Desde la perspectiva del cliente, la mediana de tiempo de respuesta mejoró levemente a 620,3 ms, alineándose con los percentiles del caso base al registrar un p95 de 757,6 ms y un p99 de 788,5 ms.

Stress Test



A una tasa promedio de 628 req/s, todas las request fueron exitosas, eliminando el 17,8% de errores que persistía con una sola API y procesando 197.324 requests. Con respecto a la

experiencia de usuario, la mediana de la transacción disminuyó a 699,4 ms (contra a los 1.587,9 ms de una sola instancia API), mientras que el p95 bajó a 1.043,3 ms y el p99 a 1.326,4 ms. La mediana del tiempo de respuesta del servidor fue de 30,9 ms. Distribuir la carga entre tres procesos independientes de Node.js evitó la saturación bajo concurrencia extrema, atacando el cuello de botella de cómputo y procesando la totalidad del tráfico sin timeouts.

3.5 Operaciones atómicas para control de concurrencia

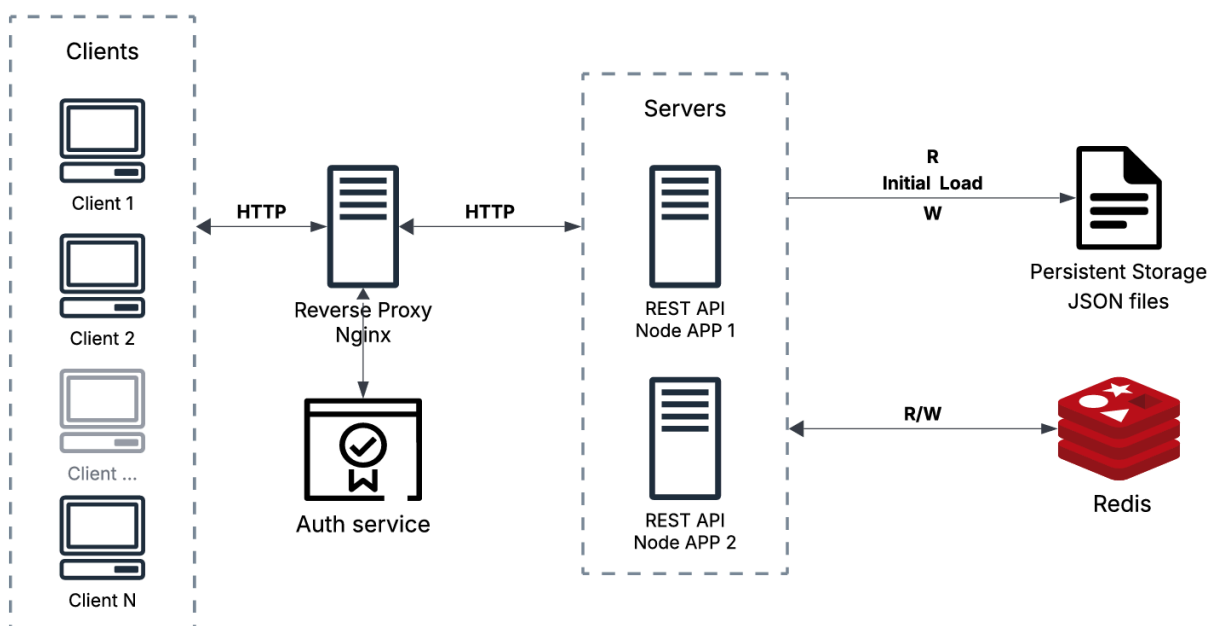
Anteriormente, analizamos la race condition: que QAs afectaba y diversas formas de resolverlo. Habíamos llegado a la conclusión, luego de varias pruebas que fuimos haciendo, que la mejor forma de mantener una performance aceptable ante un gran número de requests (que al definir el contexto del negocio lo marcamos como prioridad) era implementar un “Reservado” a la hora de realizar los exchanges donde manteníamos una muy buena performance a costa de degradar en parte la disponibilidad (ya que se podía rechazar un exchange por falta de fondos por culpa de otro exchange que finalmente no se termine realizando).

Lo negativo de los locks es que serializar todo el flujo del exchange degradaba la performance muy rápido a partir de 3 request por segundo. Las transacciones, podrían ser otra táctica viable, pero como el sistema está pensando para recibir muchas request por segundo, se generarían muchos reintentos en transacciones paralelas.

Luego del análisis del análisis de las races conditions, vimos las ventajas de migrar el estado de los datos a una base de datos Redis, pero sin concentrarnos en la race condition.

Por lo que ahora, vamos a implementar mediante [LUA SCRIPTS](#) en Redis la idea del “reservado” que confirmamos anteriormente, que es la táctica que mejor aplica a nuestro contexto de negocio.

El diagrama de componentes y conectores queda igual:



lo único que cambia ahora, es que vamos a ejecutar un script atómico dentro de redis. Lo que va a hacer este script es la reserva del dinero en la cuenta y devolvernos 1 si pudo descontar el dinero y 0 en otro caso.

Necesitamos que sea un script atómico ya que tiene dos operaciones: chequear si tiene el dinero suficiente para realizar el exchange y luego descontar el dinero.

Si las realizamos secuencialmente desde el servidor, estaríamos introduciendo una nueva race condition del mismo tipo (TOC-TOU).

Primero vamos a ejecutar los mismos escenarios que cuando analizamos la RC que nos sirven para confirmar si:

1. El código actual tiene una race condition
2. El código nuevo resuelve la race condition

Si podemos confirmar eso, vamos a analizar qué tanto degrada la performance con la táctica de reservado en Redis.

Recordemos que el escenario que utilizamos para forzar la race condition es de una sola fase de un segundo que envía 50 requests. Con los saldos base (que restauramos luego de cada prueba) y el único escenario que se puede usar en la fase, no se deberían de permitir más de 7 de esas requests. En caso contrario, significa que existe una race condition (igualmente lo confirmaremos haciendo un get al balance, viendo si las cuentas quedan en negativo)

phases:

- name: 50 request en el mismo segundo
- duration: 1
- arrivalRate: 50

scenarios:

- name: Exchange race condition (same internal counter account)
- flow:
 - post:
 - url: '/exchange'
 - json:
 - baseCurrency: 'ARS'
 - counterCurrency: 'EUR'
 - baseAccountId: 2003
 - counterAccountId: 2001
 - baseAmount: 10000000

Primero, con el caso base de este análisis (redis implementado; tres instancias del servidor; sin táctica aplicada para la race condition), chequeamos los saldos antes de ejecutar el test de carga de artillery y obtenemos:

```
{
  "id": 3,
  "currency": "EUR",
  "balance": 40000
},
```

Luego, ejecutamos artillery y ya vemos el primer mal olor. De 50 exchanges, 35 fueron confirmados al usuario con status code 200.

Esto, como ya desarrollamos anteriormente en el análisis de la race condition, es imposible que suceda ya que no hay suficiente dinero.

```
http.codes.200: ..... 35
http.codes.500: ..... 15
http.downloaded_bytes: ..... 13150
http.request_rate: ..... 20/sec
http.requests: ..... 50
```

Si volvemos a consultar ahora por las cuentas, podemos ver como la cuenta de EUR quedó negativa:

```
{
  "id": 3,
  "currency": "EUR",
  "balance": -159500
}
```

Ahora, realicemos este mismo proceso pero con el cambio del reservado realizado con Lua Scripts. Recordemos que lo único que queremos ver en este momento es si la race condition deja de existir. Posteriormente, analizaremos el impacto del cambio en la performance.

Chequeamos los balances antes de ejecutar la prueba de carga de artillery:

```
{
  "id": 3,
  "currency": "EUR",
  "balance": 40000
}
```

Vemos, que ahora los responses positivos fueron solo 7, esto nos da una buena sensación acerca de la existencia de la RC

```
http.codes.200: ..... 7
http.codes.500: ..... 43
http.downloaded_bytes: ..... 14270
http.request_rate: ..... 50/sec
http.requests: ..... 50
```

Por último, si consultamos los saldos de las cuentas, vemos como ya no quedan negativas confirmando la inexistencia de la race condition.

```
{
  "id": 3,
  "currency": "EUR",
  "balance": 100
}
```

Ahora sí, analicemos cómo impacta esta táctica en la performance general del sistema. Para esto, usaremos el test de stress que usamos en el análisis anterior donde, cuando hicimos las pruebas para tres instancias de la API teniendo la base de datos Redis implementada (pero sin aplicar ninguna solución a la race condition) pudimos mejorar la performance y procesar la totalidad de la carga sin errores. Veamos ahora, si podemos mantener ese procesamiento sin errores con el mismo escenario y la misma cantidad de instancias de la API y si la performance sufre cambios.

Stress Test:



Al repetir el stress test con tres instancias de la API utilizando Redis y scripts Lua para el acceso y modificación atómica del estado, el sistema logró procesar nuevamente la totalidad de la carga sin errores. Se procesaron 197325 requests a una tasa promedio de 632 req/s, prácticamente igual a las 197.324 requests y 628 req/s obtenidas anteriormente.

La mediana del tiempo de respuesta del servidor aumentó de 30,9 ms a 39,3 ms, lo cual es un incremento aproximado del 27,2%.

Este resultado coincide con lo que imaginamos del costo adicional introducido al tener que utilizar los Lua Scripts aunque fue mucho menor al que habíamos pensado que ocurriría antes de medirlo.

A pesar de este incremento en la mediana, el comportamiento de los casos de mayor latencia mejoró: el p95 disminuyó de 1.043,3 ms a 772,9 ms y el p99 de 1.326,4 ms a 982,6 ms.

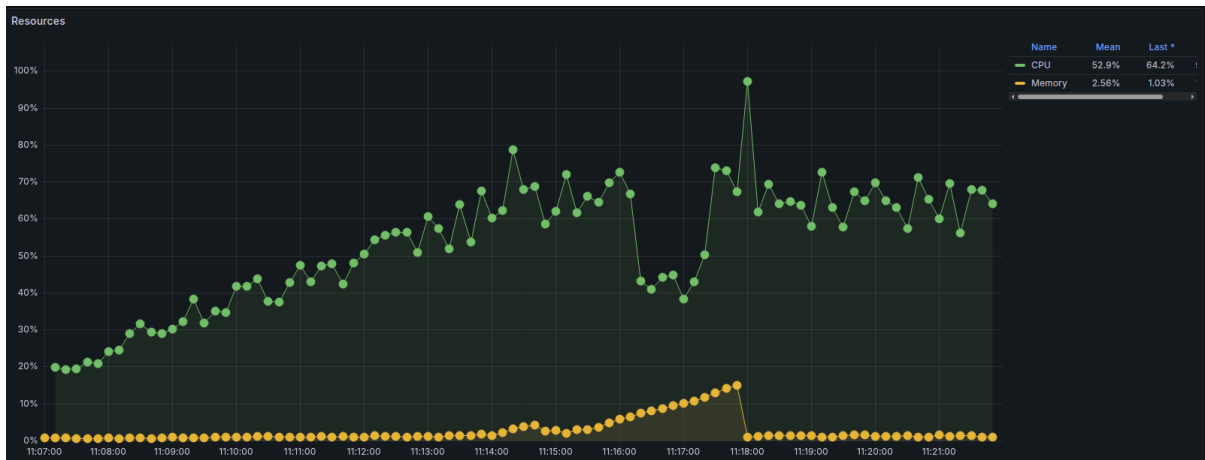
Por lo tanto, bajo este escenario de estrés, la incorporación de Redis y los scripts Lua no redujo la capacidad del sistema para procesar carga, aunque sí introdujo un overhead observable en la mediana del tiempo de respuesta.

Sin embargo, esta incorporación era necesaria ya que ahora se resolvió la condición de carrera mediante operaciones atómicas manteniendo una performance similar.

3.6 Log incremental y paginado

Como se mencionó anteriormente, un problema de la arquitectura actual es que los logs, una variable que crece infinitamente, se guardan en la memoria, y se sobrescribe constantemente el archivo log.json con dichos datos. Esto provoca que el uso de CPU y de memoria aumente de forma lineal con la cantidad de logs acumulados.

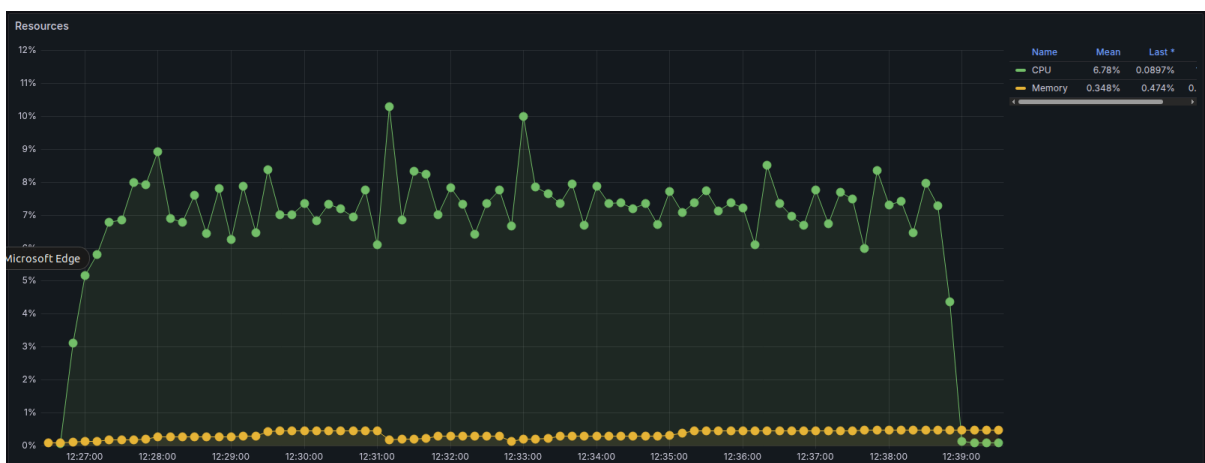
Podemos ver el gran uso de memoria que genera usando el siguiente *Stress test (ramp up: 60s, de 100 a 300 req/s, peak sustain: 660s, 300 req/s)* que genera un total de 210,019, cada una generando un nuevo log.



Como resultado del *Stress test*, el uso de la memoria creció desde un 0.06% a un 15% , y el uso de CPU creció desde 20% a alrededor de 65%, donde se mantendrá al estar constantemente reescribiendo el archivo de persistencia de logs.

Arreglamos este error cambiando la lógica con la que se maneja la memoria de los logs. En vez de guardar todos los logs en la memoria y sobrescribir constantemente el archivo log.json, ahora solamente guardamos el último segundo de logs en la memoria y lo agregamos al final del archivo antes mencionado. Además, para no empeorar la modificabilidad de la aplicación agregando funciones difíciles de leer, se decidió persistir los logs en un archivo log.ndjson que facilita el agregado y lectura de logs.

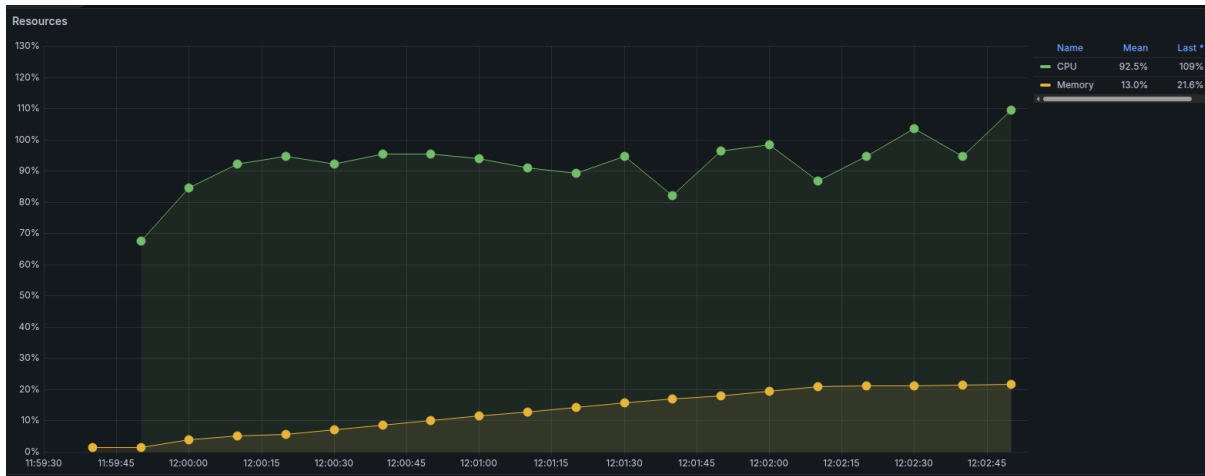
El anterior *Stress test* nos genera los siguientes resultados:



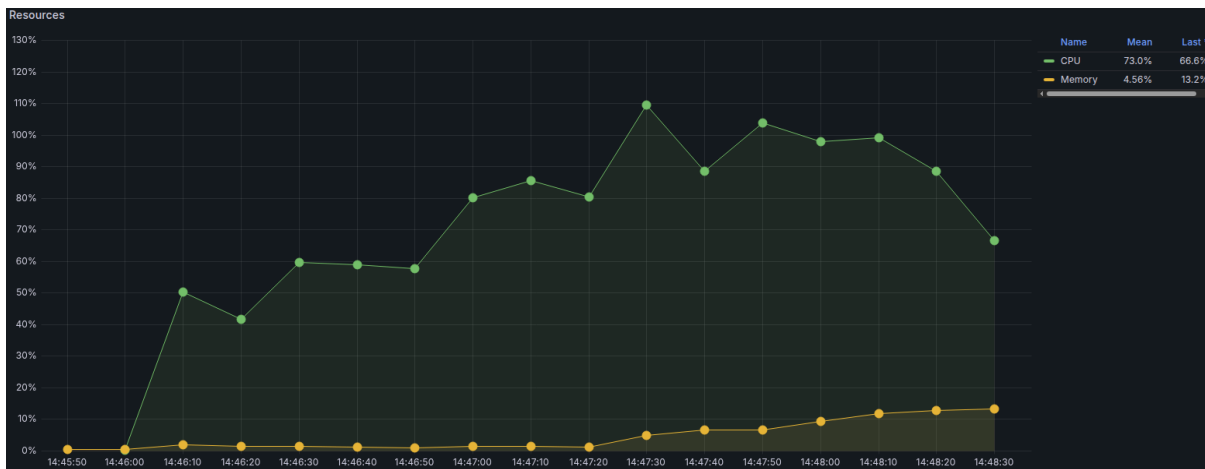
El uso del CPU crece desde 0.09% hasta alrededor del 8% y, al finalizar, vuelve a caer a 0.09% debido a que paran de haber logs que se requieran persistir. La memoria, mientras tanto, pasa de tener un uso del 0.06% a hasta 0.102% durante el periodo del test al solamente guardar el último segundo de logs.

Si bien este cambio aumenta la performance de los endpoints que generan logs como POST /exchange, empeora la performance del endpoint GET /log, al necesitar leer todo el archivo log.ndjson cada vez que es llamado, al ya no tener el valor guardado en la memoria.

Si utilizando el log obtenido previamente en el Stress test, ejecuto el siguiente *Load test (ramp up: 120s, de 1 a 5 req/s, peak sustain: 40s, 5 req/s)*, se puede observar los siguientes resultados:



En el caso base, el uso del CPU evoluciona desde alrededor del 65% hasta rondar el 95%, un aumento de alrededor del 30%.

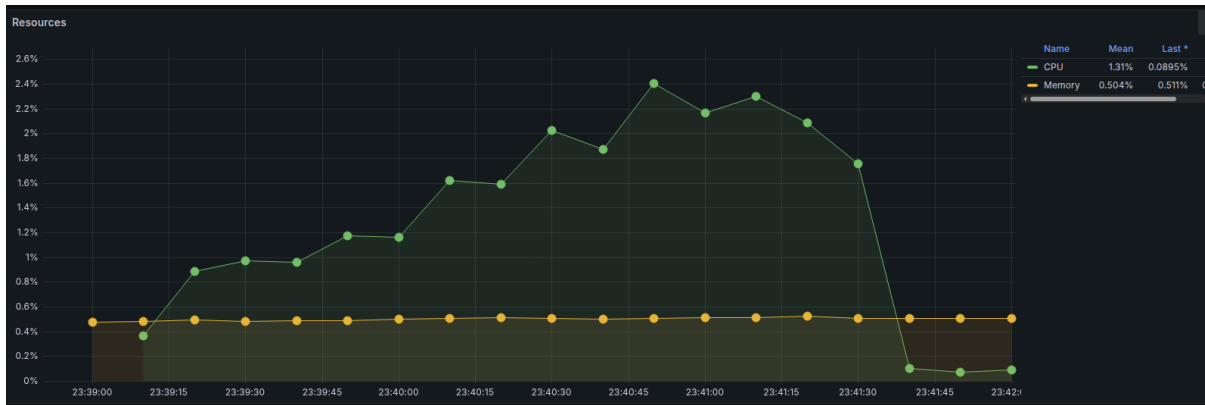


Mientras tanto, con la anterior táctica aplicada, el uso crece desde 0.08% hasta llegar a alrededor del 100%, provocando que el programa crashee debido al uso de memoria al leer el archivo log.ndjson, devolviendo el siguiente error:

```
[1:0x6e56000] 9507602 ms: Scavenge 3993.2 (4090.5) -> 3993.2 (4090.5) MB, pooled: 0 MB, 2.42 / 0.00 ms
(average mu = 0.784, current mu = 0.753) allocation failure;
[1:0x6e56000] 9507914 ms: Mark-Compact (reduce) 4095.9 (4161.3) -> 4095.8 (4128.8) MB, pooled: 0 MB, 98
.03 / 0.00 ms (+ 6.6 ms in 516 steps since start of marking, biggest step 0.7 ms, walltime since start
of marking 313 ms) (average mu = 0.761, current
FATAL ERROR: Reached heap limit Allocation failed - JavaScript heap out of memory
```

Para mejorar esta pérdida de disponibilidad, decidimos modificar el endpoint GET /log para que devuelva una respuesta paginada, y tenga los parámetros de query "page", que marca la página del log a mostrar, y "limit" que marca la cantidad de logs por página.

Si bien este cambio provoca una pérdida de usabilidad al ahora los administradores no poder comprobar todos los logs guardados, creemos que merece la pena debido a la mejora de performance que se ve en el anterior load test:



Como se puede observar, se logró bajar el uso del CPU para que se mantenga alrededor del 2%.

Creemos que el empeoramiento de usabilidad es casi negligible debido a las siguientes 2 razones:

- Gracias al sistema de autenticación y autorización que usa la aplicación, esta se debería poder asegurar de que solamente los administradores de la página puedan leer los logs. Esto significa que habría un número reducido de personas afectadas por este empeoramiento de usabilidad.
- Los logs principalmente son leídos en situaciones particulares donde los administradores necesitan revisar si ocurrió un error. Es un endpoint que se llamará intermitentemente, no uno que estaría en constante uso.

4. Bibliografía

<https://redis.io/docs/latest/develop/clients/nodejs/>

<https://github.com/redis/node-redis>

<https://redis.io/docs/latest/develop/programmability/eval-intro/>

4. 1 Uso de IA

- Ayuda con la implementación de los locks en javascript para la táctica de transacción en la Race Condition TOC - TOU
- Ayuda con la migración del estado en JSON a BDD Redis
- Como cachear los Scripts LUA con el cliente del paquete “redis” de node
- Consultas sobre cómo agregar métricas en Grafana y cómo debugearlas.
- Consultas adicionales sobre cómo mostrar las mismas métricas cuando se utilizan varios servidores.
- Ayuda con información sobre cómo leer archivos en [node.js](https://nodejs.org/) de forma paginada y de como interactuar con archivos de tipo NDJSONs.
- Ayuda para debuguear un error en la prueba de disponibilidad, el contenedor de Docker no se había reconstruido con el código nuevo después de una modificación.
- Identificación de un bug en la validación hasRate.
- Ayuda para debuguear un error en las métricas
- Ayuda configuración del docker-compose.yml